

Republic Of Iraq
Ministry Of Education and Scientific Research
Northern Technical University
Computer Engineering Technology



SMART FARM MONITOR FOR AGRICULTURAL PRODUCTION AND THE ENVIRONMENT

Topic

**A graduation project is submitted to the Computer Engineering Technology
Department in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Engineering**

BY

POLIAN AMIR POULIS

SUPERVISOR

MR. ABDULLAH M.A. HAMDOON

LECTURER

Mosul, Iraq

2022-2023

SUPERVIOSEER CERTIFICATION

I certify that the preparation of this project entitled (**Smart Farm Monitor for Agricultural Production and The Environment**), Prepared by **Polian Amir Poulis, Abdullah Ezzat Abd, Yatron Dawod Malko, Rita Elias Yono and Siva Adil Patros** was made under my supervision Computer Engineering Technology Department.

Signature:

Name: Mr. Abdullah M.A. Hamdoon

Date:

DEDICATION

To my mother, father, and all of those who have supported and encouraged me along the way, this graduation project is dedicated to you. Your love and guidance have been invaluable in helping me reach this stage in my education. Thank you for believing in me and for all of the sacrifices you have made on my behalf. I am forever grateful.

ACKNOWLEDGMENTS

I would like to extend my sincere thanks and appreciation to my supervisor, Mr. Abdullah Hamdoon , for his invaluable guidance and support throughout the research process. His expertise, knowledge, and dedication to the field of computer engineering Technology have been instrumental in shaping the direction and outcome of this project. His feedback, suggestions and encouragement have been invaluable during the research and writing process.

I am deeply grateful for the time and effort he has invested in my research, and I am grateful for his unwavering support and mentorship. This project would not have been possible without his guidance and support. Thank you, Mr. Abdullah, for everything you have done for me.

I am also grateful to the teaching staff in the Department of Computer Engineering Technology for their contributions to my education and professional development. Their patience, knowledge, and dedication have been instrumental in shaping my understanding of the field and for providing me with the tools and skills necessary to be successful. I extend my heartfelt thanks to all of you for everything you have done for me.

ABSTRACT

The objective of this project is to design and implement an intelligent robotic system for agricultural applications. This study aims to develop a smart spraying robotic system using Arduino, which can move to plants in the field and measure soil moisture, air quality and CO₂ gas level in the field, temperature and humidity in the air. The data will be sent to the computer via Wi-Fi and stored in a database, then the data will be displayed through a dedicated platform. The robot will be controlled by GPS and Bluetooth using mobile or computer, in addition to using XBEE (Zigbee) wireless communication. The benefit of this project is to control the field by taking care of the plants automatically and knowing the elements that the plant needs, in addition to detecting the risks that may be encountered in the field, such as fires.

TABEL OF CONTENTS

Supervisor certification	II
Dedication	III
Acknowledgments	IV
Abstract	V
Table of contents	VI
1 Chapter One: Introduction	1
1.1 Overview of smart farming	2
1.2 Methodology	3
1.3 Hardware Materials and components	5
1.3.1 Arduino Microcontroller (ATmega 2560 & Uno)	5
1.3.2 L298N Dual H-Bridge Controller	8
1.3.3 DC motor and wheel	9
1.3.4 HC-05 Bluetooth Module	10
1.3.5 U-Blox Neo 6M GPS Module	11
1.3.6 Digi XBee S2C Module	12
1.3.7 Compass Sensor HMC5883L	13
1.3.8 MQ135 Gas Sensor	14
1.3.9 DHT11 Humidity and Temperature sensors	18
1.3.10 Soil moisture sensor	19
1.3.11 Water Level Sensor	20
1.3.12 HC-SR04 Ultrasonic Sensor	20
1.3.13 Servo Motor	22

1.3.14 Micro DC Water Pump	24
1.3.15 Solar panel	25
1.3.16 18650 Lithium polymer Battery and Charging Module	26
1.3.17 Analog Joystick	27
1.3.18 ESP8266 Wi-Fi Module	28
1.4 Design layout and Implementation	29
1.4.1 Robot Frame	29
1.4.2 Suspension Module	30
1.4.3 Modules for Sensors	31
1.4.4 Electric System	31
1.4.5 Kinematic Model	32
1.5 Software Design Layout	35
1.5.1 Arduino Ide Software	36
1.5.2 Visual Studio	37
1.5.3 MIT App Inventor	37
1.5.4 X-CTU Software	38
2 Chapter Two: Practical Implementation	40
2.1 The aim of the chapter	41
2.2 Hardware Design flowchart	41
2.2.1 Robot Body Design	41
2.2.2 Robot Frame	43
2.2.3 Robot Sensors	43
2.3 Circuit Design	44
2.3.1 Main Circuit Diagram	44
2.3.2 Circuit Simulation	45

2.3.3 Control Unit Circuit Diagram	45
2.3.4 Actually Design	47
2.4 Software Design	47
2.4.1 Arduino IDE Algorithm	48
2.4.2 MIT App Inventor Algorithm	49
2.4.3 GUI Platform for Windows Application Algorithm	51
3 Chapter Three: Project results	52
3.1 The aim of the chapter	53
3.2 Field Trials	53
3.2.1 Incline Test	53
3.2.2 Passing over Obstacles	54
3.3 Results	55
3.3.1 Incline Test	55
3.3.2 Passing over Obstacles	55
3.3.3 Field Trials Discussion	56
3.4 Power Saving Trial	57
3.5 Data measurement accuracy	58
3.5.1 GPS reading	58
3.5.2 Compass reading	59
3.5.3 Zigbee Data Communication	59
3.5.4 Sensor reading accuracy	60
3.6 Conclusion	62
3.7 References	63

LIST OF FIGURES

Figure 1.1 Diagrams Design Block	4
Figure 1.2 Arduino Mega 2560.	6
Figure 1.3 Arduino Uno	6
Figure 1.4 L298n Dual H-Bridge Controller Module	9
Figure 1.5 Dc Motor with Wheel	10
Figure 1.6 Hc-05 Bluetooth Module	11
Figure 1.7 GPS U-Blox 6m Module	12
Figure 1.8 Digi Xbee S2c Module	13
Figure 1.9 Xbee Arduino Shield	13
Figure 1.10 Hmc5883l Compass Sensor	14
Figure 1.11 Mq-135 Gas Sensor	15
Figure 1.12 Sensitivity Characteristics Mq-135	15
Figure 1.13 Sensor Circuit	16
Figure 1.14 Circuit Simplified	16
Figure 1.15 Combined R_s and R_l	17
Figure 1.16 Dht11 Humidity and Temperature Sensors	18
Figure 1.17 Soil Moisture Sensor	19
Figure 1.18 Water Level Sensor	20
Figure 1.19 Movement of The Ultrasonic Signal from The Transmitter to The Receiver	21
Figure 1.20 The Ultrasonic Wave Pattern	21

Figure 1.21 Ultrasonic Wave Move with An Angle	21
Figure 1.22 Dc Servo Motor	23
Figure 1.23 Input Pulse of Servo Motor	23
Figure 1.24 Relationship Between Pulse and Direction of Servo Motor	24
Figure 1.25 Dc Water Pump	25
Figure 1.26 Amorphous Solar Panel	26
Figure 1.27 18650 Lithium Battery	27
Figure 1.28 18650 Charger BMS Protection Board	27
Figure 1.29 Analog Joystick	28
Figure 1.30 Esp8266 Wi-Fi Module	28
Figure 1.31 Robot Suspension Module	30
Figure 1.32 Plastic Storage Containers Boxes	31
Figure 1.33 Mobile Robot Geometry	32
Figure 1.34 Arduino IDE Software Program	36
Figure 1.35 MIT App Inventor Design Editor	38
Figure 1.36 MIT App Blocks Editor	38
Figure 1.37 X-CTU Software	39
Figure 2.1 Hardware Design Flowchart	41
Figure 2.2 Metal Plate	42
Figure 2.3 Hard Disk Drive Box	42
Figure 2.4 Robot Frame and Tires	43
Figure 2.5 Main Circuit Diagram	44
Figure 2.6 Circuit Simulation Diagram	45
Figure 2.7 Control Unit Circuit Simulation Diagram	46

Figure 2.8 Control Unit of the Robot	46
Figure 2.9 Real Pictures of The Robot	47
Figure 2.10 Arduino Ide Algorithm Flowchart	48
Figure 2.11 MIT App Inventor Algorithm Flowchart	49
Figure 2.12 MIT App Inventor Application Design	49
Figure 2.13 GUI Platform Algorithm Flowchart	51
Figure 3.1 The incline Test in The Robot	54
Figure 3.2 The obstacles Test in The Robot	55
Figure 3.3 The Robot During the Test	56
Figure 3.4 GPS obtained data using Arduino serial monitor	58
Figure 3.5 Calibrating the compass using Arduino	59
Figure 3.6 The Data Results That Obtained	61
Figure 3.7 MQ-135 and DHT-11 reading data	61
Figure 3.8 Rain sensor and soil moisture sensor reading data	61

=====

LIST OF TABELS

Table 1. 1Arduino Mega 2560 Specifications	7
Table 1.2 Arduino Uno Specifications	7
Table 1.3 Pins Assignment for L298	8
Table 1.4 Dc Motor Specifications	10
Table 1.5 Dht11 Sensor Specifications	18
Table 1.6 Water Level Sensor Specifications	20
Table 1.7 Dc Water Pump Specifications	25
Table 1.8 Esp8266 Wi-Fi Module Specifications	29

Chapter One

Introduction

Chapter One: Introduction

1.1 Overview of smart farming

Agriculture plays a vital role throughout human history from ancient years until now, as it is essential for the survival of human's species. During this period, it has seen many evolutions, from the domestication of animals and plants a few thousands of years ago, to the systematic use of crop rotations and other advancements in farming practices a few hundred years ago. Human-made fertilizers and pesticides were the last innovation a few decades ago. Nowadays, we are experiencing a new evolution in the agriculture sector, called Smart Farming [1], which is based on Information and Communications Technologies (ICT). Smart Farming is aiming to increase productivity and improve the quality of the final product while reducing cost production. To achieve that, a range of recent technologies are used, including Unmanned Aerial Vehicles (UAVs) [2], Unmanned Ground Vehicles (UGVs) [3], [4], Image Processing, Machine Learning, Big Data, Cloud Computing, and Wireless Sensor Networks (WSNs).

Smart technologies in agriculture will boost the production of farm crops and livestock since autonomous systems will be able to control actuators effectively, improve the utility, control resource usage, and ensure products conform to market requirements while maximizing profit and minimizing the cost of production [5]. Smart farming refers to the use of technologies such as IoT for collection of weather data, monitoring of crop's growth, early detection of crops diseases, prevention of crops wastage dues to effective harvesting of crops, monitoring of livestock's behavioral patterns, animal location within and outside the farms, increase of production

for both crops and livestock using of application of diverse and improved farming strategies, techniques for crop planting and harvesting, and the use of mechanized tools for agriculture. During the prehistoric age, farming was practiced using sticks, sickle, hand gathering of crops, and hunting of animals. According to [6], agriculture has changed. Farmers can now monitor their farms remotely from their smartphones and control devices. Farmers cultivate crops using seeds that have been genetically modified to prevent disease and infestation on the farm. These seeds also help improve the quality of the crops produced and boost the volume of the harvest. It can be deduced from [6], that the improved quality of crops has reduced food scarcity across the globe. In this study, we will discuss the new technologies available in our modern era, which can be used in the manufacture of a multi-purpose smart rover that can perform most of the tasks in the field, such as planting seeds and watering crops when needed, as well as detecting fires inside the field if any, and many other tasks.

1.2 Methodology

The solar panel is connected to the battery for storing energy and further it is given to power supply charging circuitry which is providing +5 V for Arduino board and +12 V supply for driving DC motors using driver motor module. The Microcontroller is the brain of the system, as seen in Fig 1.1. It collects data and generates outputs in accordance with the program's instructions. The GPS will determine the location and send data to the mobile phone via Bluetooth, so that the starting point and the end point are determined, then a waypoint is made between the two points. Then The moisture sensor will start detecting moisture, and if water is required for the land, the water pump is turned on; otherwise, the water pump is turned off. This project included four categories of work:

- i. Measuring the CO2 level, rain level, temperature, and humidity in the air
- ii. Determine the soil's moisture content. When the ground is dry, turn on the water pump automatically. When there is enough water in the ground, the sensor will automatically detach, and the water pump will be turn off.
- iii. Killing weeds in the field by using laser technology
- iv. Sending data to the computer using Wi-Fi module

This chapter will go through hardware components and the mechanical design of the vehicle as well as the languages used in programming.

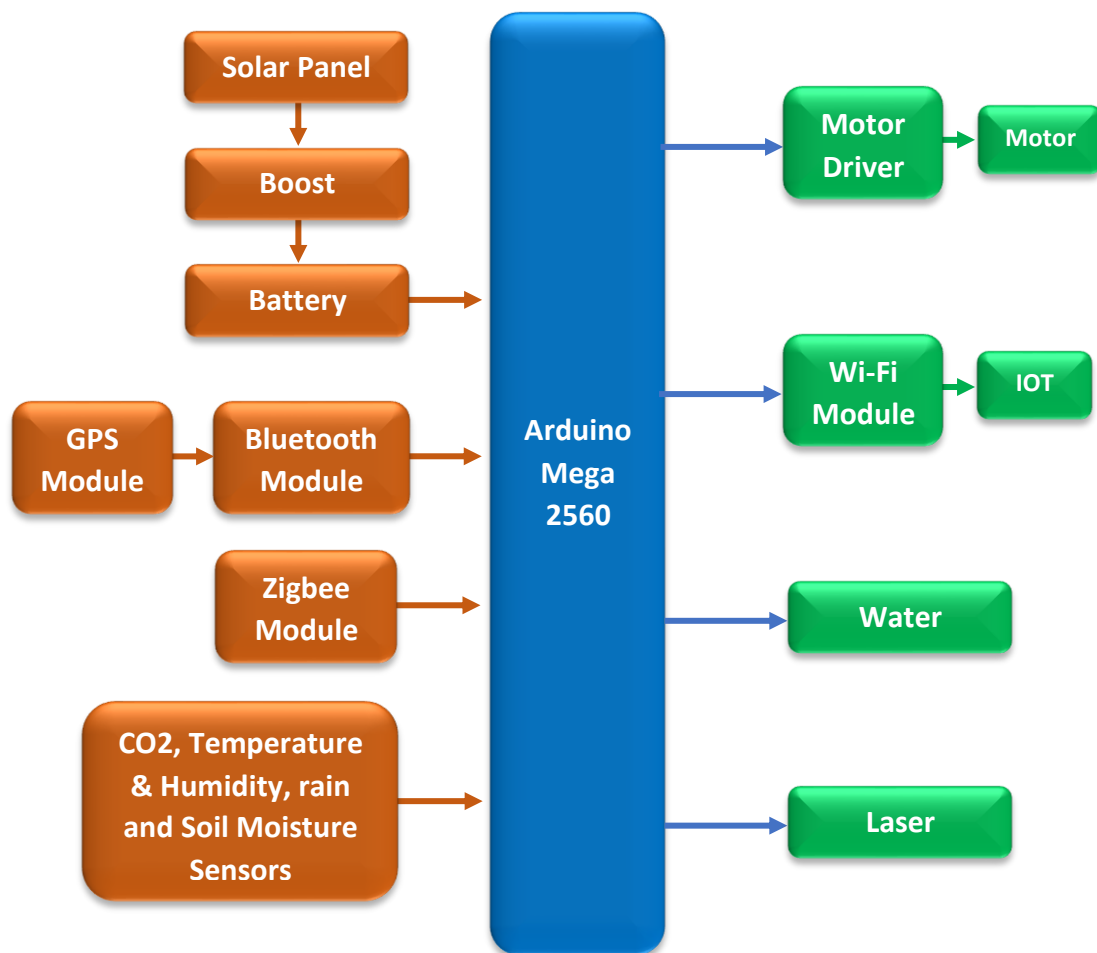


Figure 1.1 Diagrams Design Block

1.3 Hardware Materials and components

The system design consists of Arduino, motors, and sensors. Each piece will be briefly explained in following sections.

1.3.1 Arduino Microcontroller (ATmega 2560 & Uno)

ARDUINO MEGA 2560 is an ATmega2560 based microcontroller board. Having 54 digital I / O pins including 15 pins can be used as Pulse Width Modulation (PWM) output, 16 pins as analog input, 4 pins as UART (hardware serial port), using 16 MHz crystal oscillator, USB connection, power supply connector with Jack type, ICSP header, and reset button [7]. It is an open-source single-board microcontroller developed from the Wiring Platform and made to facilitate the use of electronics, namely Atmel AVR-type microcontroller processors, in various applications. IC). The microcontroller itself is a chip or IC (Integrated Circuit) that can be programmed by a computer. The purpose of embedding a program on a microcontroller is so that the electronic circuit can read the input, process the input, and produce the desired output. Thus, the microcontroller acts as a brain that controls the process of plugging in and out of electronic circuits Write [8]. We used Arduino Mega because it has many pins and 5V ports so we can connect more than one sensor and supply power to these sensors. The minimum standard for this template, please follow this guide must exist for a manuscript.

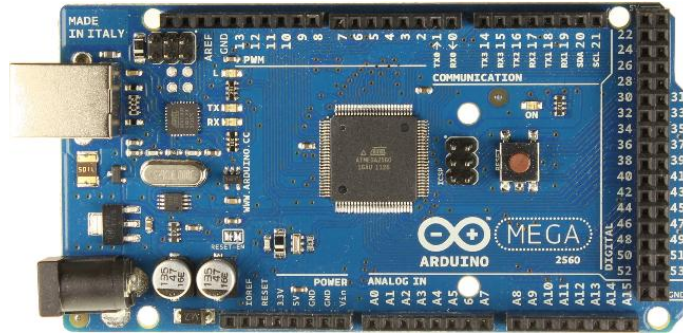


Fig1.2 Arduino Mega 2560.

Arduino Uno is a microcontroller board based on the ATmega328P shown in Fig1.3. It has 14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz quartz crystal, a USB connection, a power jack, an ICSP header and a reset button. It contains everything needed to support the microcontroller; simply connect it to a computer with a USB cable or power it with an AC-to-DC adapter or battery to get started. In this paper, the Arduino microcontroller is very well-suited to drive the PWM signal for the DC motor for the improvement of the output response for the DC motor position control system [9].



Figure 1.3 Arduino Uno

Table 1.1 Arduino Mega 2560 Specifications

Microcontroller	ATMega 2560
Operating Voltage	5 V
Input Voltage (recommended)	7-12 V
Input Voltage (limit)	6-20 V
Pins I/O Digital	54 (of which 14 provide PWM outputs)
Analog Input Pins	16
DC Current Per I/O Pin	40 mA
DC Current For Pin 3.3 V	50 mA
Flash Memory	256 KB of which 8 kb is used by the bootloader
SRAM	8 KB
EEPROM	4 KB
Clock Speed	16 MHz

Table 1.2 Arduino Uno Specifications

Microcontroller	ATMega 328P
Operating Voltage	5 V
Input Voltage (recommended)	7-12 V
Input Voltage (limit)	6-20 V
Pins I/O Digital	14
Analog Input Pins	6
DC Current Per I/O Pin	20 mA
DC Current for Pin 3.3 V	50 mA
Flash Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz

1.3.2 L298N Dual H-Bridge Controller

The L298N H-bridge IC shown in Fig1.4 that can allow to control the speed and direction of two DC motors. This module can be used with motors that have a voltage of between 5 and 35V DC with a peak current up to 2A. The module has two screw terminal blocks for the motor A and B, and another screw terminal block for the Ground pin, the VCC for the motor and a 5V pin which can either be an input or output. Pin assignments for the L298N dual H-Bridge Module is shown in table 1.3. The digital pin assigned from HIGH to LOW or LOW to HIGH is used IN1 and IN2 on the L298N board to control the direction. And the controller output PWM signal is sent to ENA or ENB to control the position. The forward and reverse speed or position controlling for the motor is done by using PWM signal. Then using the `analogWrite()` function and send the PWM signal to the Enable pin of the L298N board, which actually drives the motor [9].

Table 1.3 Pins assignment for L298

Out 1: Motor A lead out
Out 2: Motor A lead out
Out 3: Motor B lead out
Out 4: Motor B lead out
GND: Ground
5V : 5V input
ENA: Enables PWM signal for Motor A
IN1: Enable Motor A
IN2: Enable Motor A
IN3: Enable Motor B
IN4: Enable Motor B
ENB: Enables PWM signal for Motor B

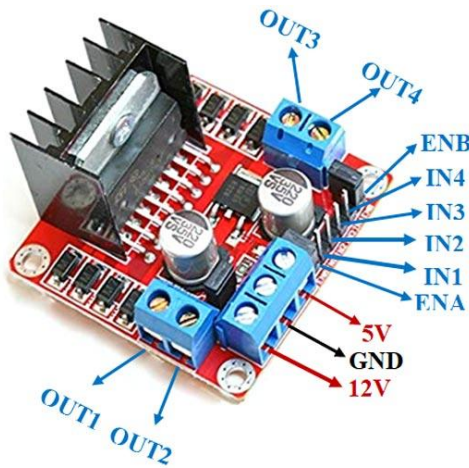


Fig 1.4 L298N dual H-Bridge Controller Module

1.3.3 DC motor and wheel

A brushless DC (BLDC) motor shown in Fig1.5 is a synchronous electric motor powered by direct-current (DC) electricity and having an electronic commutation system, rather than a mechanical commutator and brushes. In BLDC motors, current to torque and voltage to rpm are linear relationships. Some of the problems of the brushed DC motor are eliminated in the brushless design. A DC hobby motor with a 48:1 gearbox and a 66mm wheel are selected for this system. Ideal for small robotics projects, such as a Simple Bot or other car type design. Easy to control using either N-DRIVE module or a PWM output from Arduino for single-direction variable speed, or H-Bridge shield for forward / reverse variable speed. The gearbox includes a press-fit axle that allows the wheel to be mounted on either side [10]. DC motor Specifications shown in table 1.4.

Table 1.4 DC motor Specifications

Working voltage: 3-6Vdc
Gearbox ratio: 48:1
Motor size: 70x22x18mm
Motor weight: 50g
Wheel diameter: 66mm
Wheel weight: 12g.



Fig 1.5 DC motor with wheel

1.3.4 HC-05 Bluetooth Module

The HC-05 motor shown in Fig1.6 is a 6 pin IC where the TX and RX pin is connected to the RX and TX pin of the Arduino board respectively. The potential divider circuit used is to reduce the Arduino 5V potential to 3.3V for proper transmission and reception between the Arduino and the Bluetooth Module. We have to make sure that the baud rate of the Bluetooth module is synchronized with that of the Arduino so that there is no loss of

data and proper communication is achieved [11]. HC-05 module is an easy-to-use Bluetooth SPP (Serial Port Protocol) module, designed for transparent wireless serial connection setup. Serial port Bluetooth module is fully qualified Bluetooth V2.0+EDR (Enhanced Data Rate) 3Mbps Modulation with complete 2.4GHz radio transceiver and baseband. It uses CSR Bluecore 04-External single chip Bluetooth system with CMOS technology and with AFH (Adaptive Frequency Hopping Feature) [12].

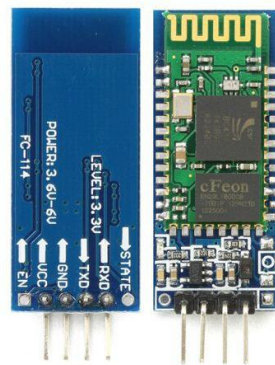


Fig 1.6 HC-05 Bluetooth Module

1.3.5 U-Blox Neo 6M GPS Module

The GPS U-Blox 6m module as shown in Fig1.7 is a kind of stand-alone GPS receiver that features high performance as a positioning machine [13]. With optimized architecture, power, and memory this module is perfect for devices that use the battery as a resource with limited cost and space making it very suitable to use on CanSat. By having 50 channels of positioning engines, it can accelerate Time-ToFirst-Fix (TTFF) for less than 1 second. This module has an outer receiving antenna and implicit EEPROM. To get crude GPS information it simply needs to begin a sequential correspondence with the GPS module utilizing Software Serial. When Arduino or any microcontroller and GSM module on the first Stage.

Subsequently, we need to utilize just the C language adversary inserted programming. The following stage is to arrange preparation. You ought to get a lot of data in the GPS standard language, NMEA. Each line you get in the sequential screen is a NMEA sentence. NMEA represents National Marine Electronics Association, and in the realm of GPS, it is a standard information group upheld by GPS producers [14].

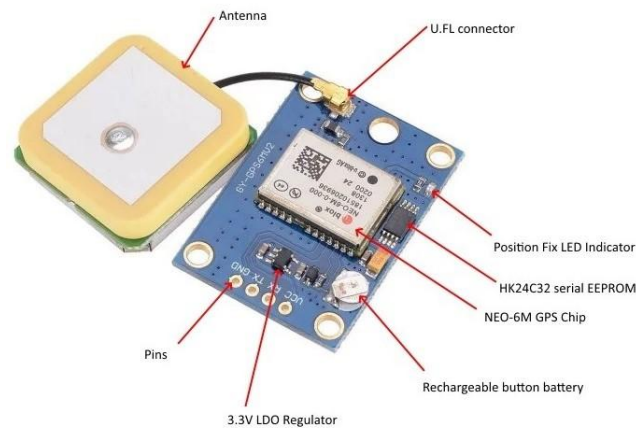


Fig 1.7 GPS U-Blox 6m module

1.3.6 Digi XBee S2C Module

The module Xbee used in this project is from Digi XBee S2C embedded RF transceiver modules to provide wireless connectivity to each other by Zigbee protocol. To configure this module, the XCTU was used to test the connection between Coordinator and Router by AT commands and MAC address. This connection can reach 60 m indoor and 1200m outdoor or RF line of sight range. Fig 8. show the version S2C of Xbee which is used in this system [15] Fig 1.9 Show XBee Shield for Arduino.

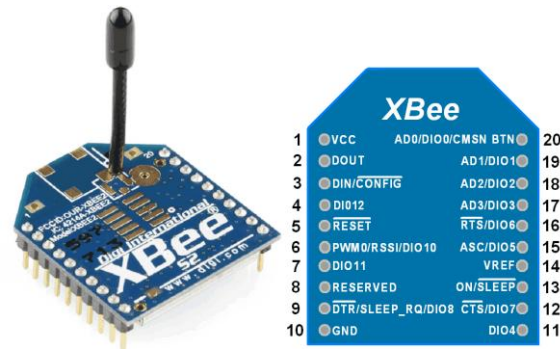


Fig 1.8 Digi XBee S2C Module

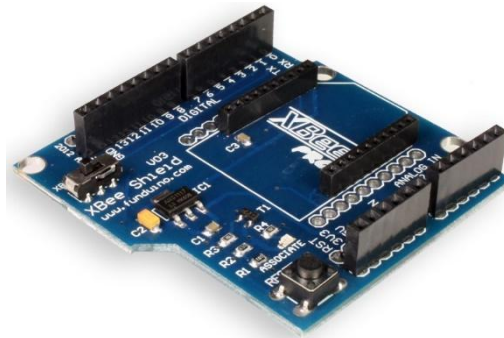


Fig 1.9 XBee Arduino Shield

1.3.7 Compass Sensor HMC5883L

A compass is a navigation tool for determining the direction of a magnetic pointer arrow that is free to accurately align itself with the earth's magnetic field. Compass provides a reference for certain directions, so it is very helpful in the field of navigation. The cardinal directions that it shows are north, south, east, and west. HMC 5883L is a sensor that is used as shown in Fig1.10 to show the cardinal directions, or it is also known as a digital compass. This sensor uses the main component in the form of IC HMC5883L which is a 3-axis digital compass IC which has an interface in the form of I2C pins. HMC5883L has a high-resolution magneto-resistive sensor HMC118X series, plus ASIC with amplification content, automatic

degaussing strap driver, offset cancellation and a 12-bit ADC that allows compass accuracy of 1 to 2 degrees. This module is commonly used for automatic navigation systems, mobile phones, netbooks and personal navigation devices. This module has 5 pins, including VCC, Gnd, SDA, SCL, and DRDY [14]. There are three magnetoresistive elements used by the HMC5883L sensor, one of which is that it can change the resistance proportional with the strength of the magnetic field along its axis. It should be noted that the way this sensor works is by using the earth's magnetic field as a reference for the magnetic field points on the north and south sides. An important point to note is that the sensitivity of each individual magnetoresistive element is a component that is parallel between the magnetic field and the element's axis [16].

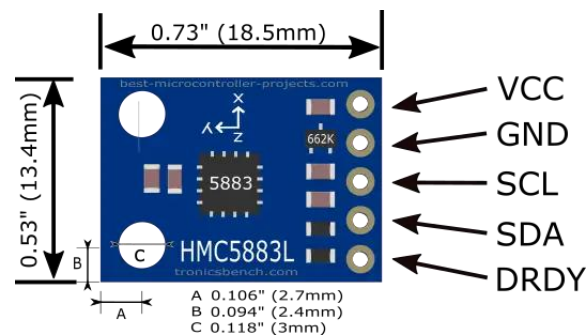


Fig 1.10 HMC5883L Compass Sensor

1.3.8 MQ135 Gas Sensor

The (MQ-135) gas sensor as shown in Fig1.11 utilized for air quality and is detecting or measuring NO_x, Nh₃, CO₂, Benzene, Alcohol, Smoke. The (MQ-135) sensor comes with a digital pin that makes this sensor work with or without microcontroller, which is good use when trying to discover one particular gas. To measure the gasses in PPM (part per million) the analog

pin needs to be used. The analog TTL is operated at 5 volts and so can be used with most common microcontrollers. According to Sensitivity Characteristics (MQ-135) as shown in Fig1.12 the resistance ratio R_s/R_0 . Resistances R_L can be identified if the resistance sensor changes according to gas concentration, & (R_0) resistance sensor presence concentration is known in fresh air [17].



Fig 1.11 MQ-135 Gas Sensor

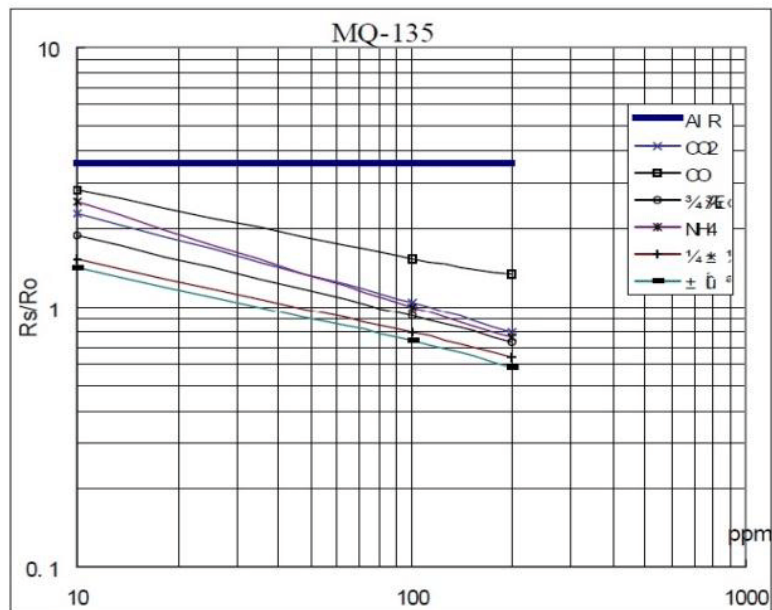


Fig 1.12 Sensitivity Characteristics MQ-135

Derive a formula to find R_S

From the electrical circuit shows in Fig1.13, R_S is resistor between (pin A and pin B) and resistor R_L [9] where:

- (H) Pins: Out of the two H pins, one pin is connected to supply and the other to ground.
- (A) Pins: pins (A and B) are interchangeable. They connected to the Supply voltage.
- (B) Pins: pins (A and B) are interchangeable. One of them will serve as the output while the other will be pulled to the ground.

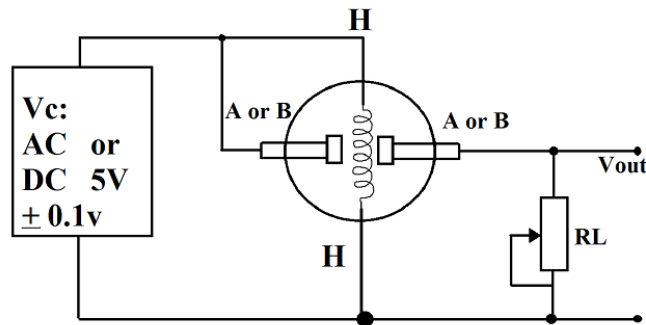


Fig 1.13 Sensor circuit

By simplify circuit as shown in Fig1.14

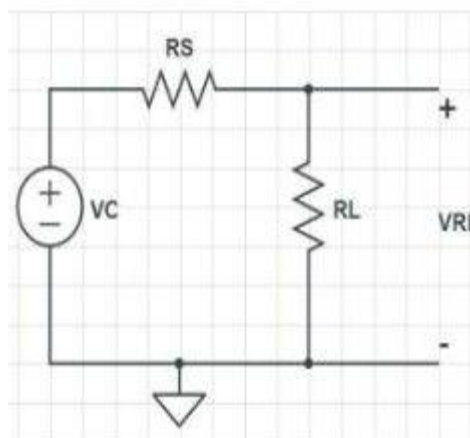


Fig 1.14 Circuit Simplified

Combine resistors RS and RL in series as shown in Fig1.15

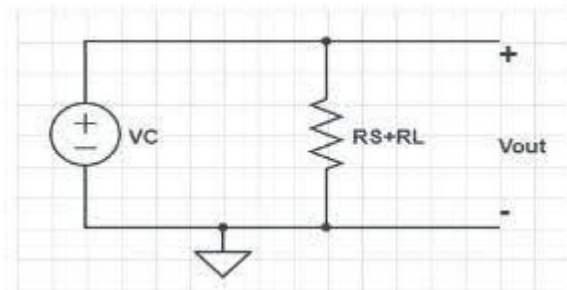


Fig 1.15 Combined RS and RL

$$I = V / R \dots\dots\dots \text{"Ohm's Law"}$$

$$V = VC \dots\dots\dots$$

$$R = RS + RL_1$$

$$I = VC / (RS + RL_1)$$

$$V = I \times R$$

$$V_{RL} = [VC / (RS + RL_1)] \times RL_1$$

$$V_{RL} = (VC \times RL_1) / (RS + RL_1)$$

$$V_{RL} \times (RS + RL_1) = VC \times RL_1$$

$$(V_{RL} \times RS) + (V_{RL} \times RL_1) = VC \times RL_1$$

$$(V_{RL} \times RS) / V_{RL} = [(VC \times RL_1) - (V_{RL} \times RL_1)] / V_{RL}$$

$$RS = [(VC \times RL) / V_{RL}] - RL$$

This formula (RS) is used for different gassses [17].

1.3.9 DHT11 Humidity and Temperature sensors

DHT11, shown in Fig1.16, is a temperature and humidity sensor that has four pins- VCC, GND, Data Pin and a not connected pin. A pull-up resistor of 5k to 10k ohms is probed for communication between sensor and microcontroller. DHT11 sensor consists of a capacitive humidity sensing element and a thermistor for sensing temperature. The humidity sensing capacitor has two electrodes with a moisture holding substrate as a dielectric between them. Change in the capacitance value occurs with the change in humidity levels. The IC measures, processes these changes resistance values and change them into digital form. For measuring temperature this sensor uses a Negative Temperature coefficient thermistor, which causes a decrease in its resistance value with increase in temperature [18]. DHT11 sensor Specifications shown in table 1.5.

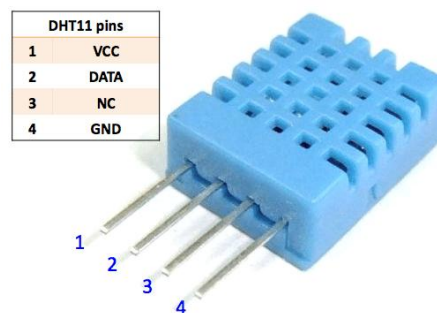


Fig 1.16 DHT11 Humidity and Temperature sensors

Table 1.5 DHT11 sensor Specifications

Measurement range	Temperature	0 to 50°C
	Humidity	20 to 90 % RH
Accuracy	Temperature	±2 %
	Humidity	±5 %

Humidity is defined as the amount of water vapor contained in air. Usually, it is expressed as absolute humidity, dewpoint and relative humidity. The sensor used in this thesis project, DHT11, is designed to measure humidity in terms of relative humidity (RH). Relative humidity (RH) is the ratio of the amount of water vapor content of the air to the saturated moisture level at the same pressure or temperature:

$$RH = \frac{\rho_w}{\rho_s} \times 100\%$$

Where RH is relative humidity, ρ_w is the density of water vapor, and ρ_s is the density of water vapor at saturation [19].

1.3.10 Soil moisture sensor

Uses the moisture sensor in the soil is used by implanting two electrodes (cathode anode) in the agricultural soil and after the water flows into the soil, these electrodes are affected electrically and send signals representing the electric voltage difference to Arduino device and then to the PC to close or open the valves to ensure flow or close the water. Fig1.17 shows the electrical circuit work. The soil moisture sensor consists of three pins (VCC, GND, SIG), VCC is the regulated DC supply voltage, GND is the ground, and the third (SIG) sends the signal to one of Arduino's serial ports for reading soil moisture [20].

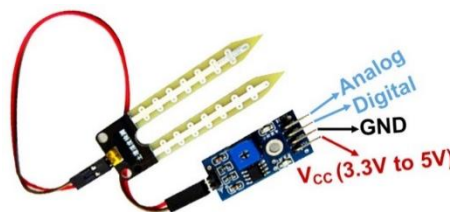


Fig 1.17 Soil moisture sensor

1.3.11 Water Level Sensor

The 3-5V DC water level sensor shown in Fig1.18 uses no more than 20 mA of current during operation. The compact device is an easy to use, cost effective high-level recognition sensor that uses a series of exposed traces on parallel wires to measure the water volume in order to determine the water level. Its analog signal conversion and output analog values make it easily compatible with the Arduino Uno [21]. Water Level sensor Specifications shown in table 1.6.

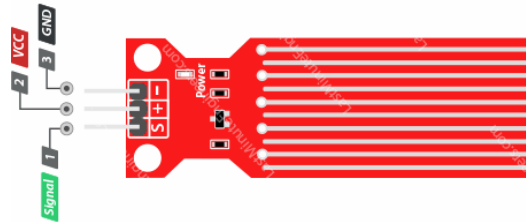


Fig 1.18 Water Level Sensor

Table 1.6 Water level sensor Specifications

S(Signal) Pin	Analog Pin
+ (VCC) Pin	3.3V - 5V
- (GND) Pin	Ground Pin

1.3.12 HC-SR04 Ultrasonic Sensor

The ultrasonic rangefinder HC-SR04 where the transmitter and the receiver are designated as T and R, respectively. The rangefinder generates sound waves at a frequency of 40 kHz. The sound waves reflect from the object and return to the receiver, the sensor gives the information about the time, which was demanded to sound waves for propagation from the sensor to the object and back [22]. This process can be clearly seen in Fig1.19.

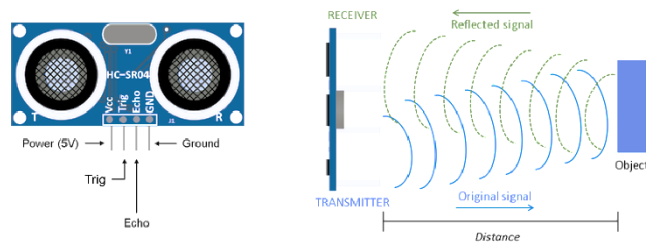


Fig 1.19 Movement of the ultrasonic signal from the transmitter to the receiver

The ultrasonic signal is propagated by a wave directed at an angle of 30° . The direction of propagation of the ultrasonic signal from the transmitter is shown in Fig1.20. The most effective measuring angle is 15° . External objects falling under this measurement angle interfere with the determination of the distance to the desired object [22] Fig1.21 shows the movement of the ultrasonic wave from the transmitter to the receiver at an angle.

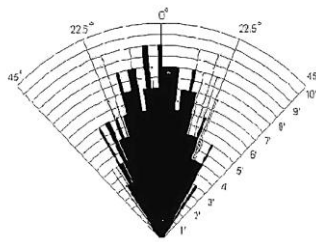


Fig 1.20 The ultrasonic wave patterns

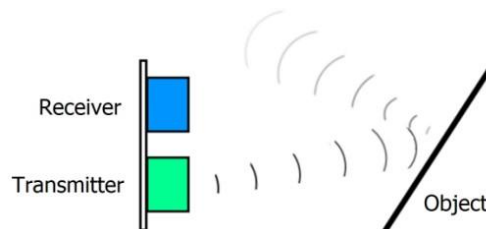


Fig 1.21 Ultrasonic wave Move with an angle

The readings of the ultrasound range finders are not affected by sunlight or the color of objects, as is the case with infrared sensors. The ultrasonic wave is reflected from almost any surface, even transparent, but it can be difficult to determine the distance to fluffy or small objects. In addition, the readings are affected by the angle of incidence of the wave. If the sensor is directed perpendicular to the object, the measurements will be most accurate. In addition, if the angle of incidence is too large, then the wave, reflected from the object, does not enter the receiver, which will lead to an incorrect measurement. To calculate the distance from the sensor to the object. The sensor itself does not count anything on its own, but only gives an impulse of certain duration. All calculations must be made by the microcontroller [22].

1.3.13 Servo Motor

A servo motor shown in Fig1.22 is an electromechanical device in which the electrical input determines the position of the motor armature. It is actually an assembly of four things: a normal DC motor, a gear reduction unit, a position-sensing device and a control circuit. Servo motors are used extensively in the robotics industry and radio-controlled cars. The implemented types in modern servo systems are AC servo motors based on induction motor designs, DC servo motors based on DC motor designs, DC brushless motors and AC brushless servo motors based on synchronous motor designs. These motors are working in closed loop control systems where the programmed position of motion and velocity feedback controllers are required [23].

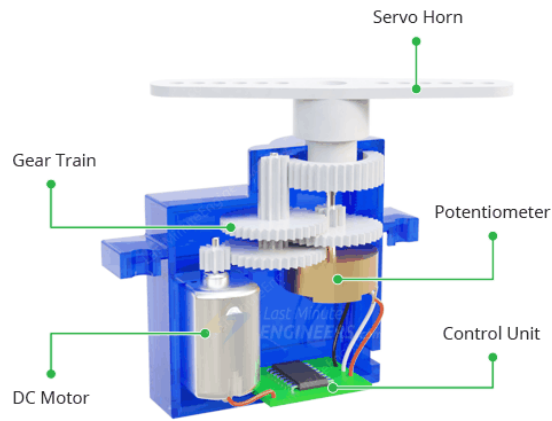


Fig 1.22 DC Servo Motor

Servos are controlled by a pulse of variable width. The sent signal of this input pulse is characterized by a minimum pulse, maximum, and a repetition rate as seen in Fig1.23. Given the rotation constraints of the servo, neutral is defined to be the position where the servo has exactly the same amount of potential rotation in a clockwise direction as it is in a counter clockwise direction. The angle is determined by the duration of applied pulse to the signal wire which is called PWM or Pulse Coded Modulation. The servo should detect a pulse every 20ms. The length of the pulse will determine how far the motor turns. For example, a 1.5ms pulse will make the motor turn to a 90-degree position (neutral position). The position pulse must be repeated to instruct the servo to stay in position [23].

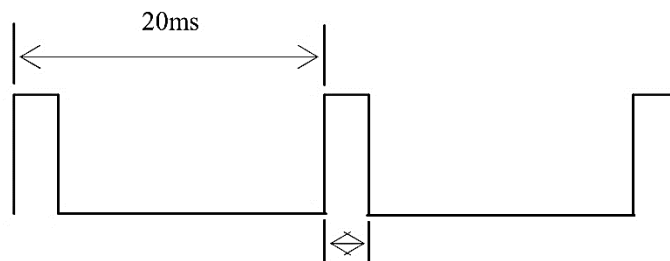


Fig 1.23 Input pulse of servo motor

When a pulse is sent to a servo that is less than 1.5ms, the servo rotates to a position and holds its output shaft some number of degrees counterclockwise from the neutral point. When the pulse is wider than 1.5ms, the opposite operation occurs. The minimal width and the maximum width of pulse that will command the servo to turn to a valid position are functions of each servo. Generally, the minimum pulse will be about 1ms wide (some servo is 0.5ms) and the maximum pulse will be 2ms wide (some servo is 2.5ms). The servo motor operates in the range of 5 % to 10 % of duty cycles [23]. Fig1.24 shows the relationship between pulse and direction of servo motor.

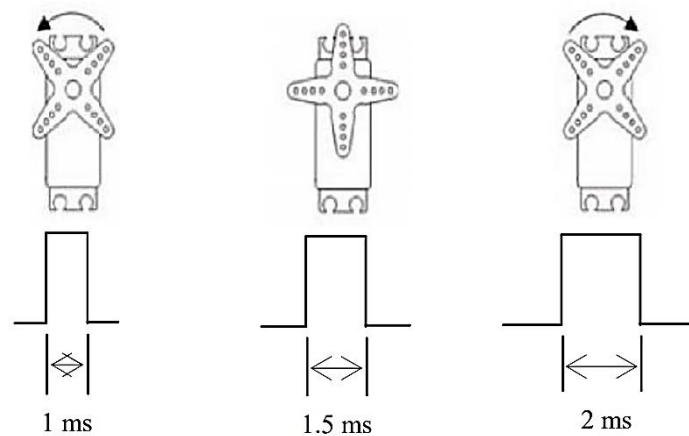


Fig 1.24 Relationship between pulse and direction of servo motor

1.3.14 Micro DC Water Pump

Micro Water Pump Shown in Fig1.25 that operates on DC 3V 5V. The Pump works by suction via the inlet and releases the water through the outlet terminal. It can be used in aquariums as exhaust systems and water flow fountains [24]. DC Water Pump Specifications shown in table 1.7.

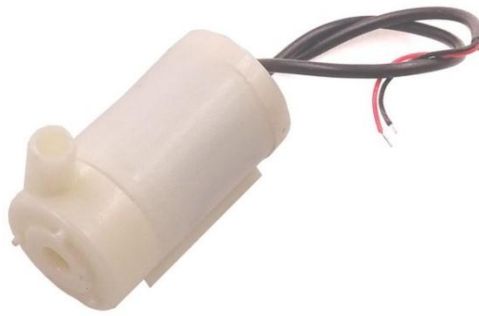


Fig 1.25 DC Water Pump

Table 1.7 DC Water Pump Specifications

Input Voltage	DC 3V-5V
Flow Rate	1.2-1.6 L/min
Operation Temperature	80°C
Operating Current	0.1-0.2A
Suction Distance	0.8 meter (Max)
Outside diameter of water outlet	7.5 mm
Inside diameter of water outlet	5.0 mm
Diameter of water Inlet	5.0 mm
Wire Length	200 mm
Rubber tube length	1 meter
Size	45 x 30 x 25 mm
Weight	30g

1.3.15 Solar panel

The solar panel works based on the principle of photovoltaic effect which converts the solar energy into electrical energy. There are various types of panels available like monocrystalline, polycrystalline, amorphous, and hybrid. In this project we use a 40W 12V amorphous solar panel is considered. The Amorphous solar panel was shown in above Fig1.26 The

cost and silicon requirement of the amorphous panel is low when compared with the other types of solar panel [25].

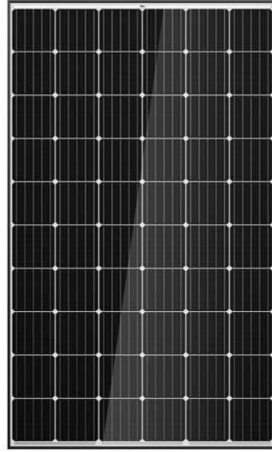


Fig 1.26 Amorphous solar panel

1.3.16 18650 Lithium polymer Battery and Charging Module

Lithium-polymer batteries as shown in Fig1.27 are high-powered lithium batteries used for electric vehicles. Lithium- polymer technology uses a positive and negative electrode, but with a dry solid, porous chemical, rather than a liquid. The lithium-polymer battery cell holds more than 70% of its capacity between 3.2 and 3.7. Lithium-polymer batteries are flexible, and leak-resistant with high conductivity electrolyte film (SPE) have the potential to deliver the best power available for rechargeable batteries. Lithium-type batteries are usually arranged to produce the desired voltage and capacity. For the voltage and current of this battery array to be balanced, a system is required that can set it called a Battery management system (BMS). The average voltage of a lithium battery is 3.7 V, so it needs to be composed of 3S (series) to produce 12V. Fig1.28 is the BMS 3s scheme of lithium-polymer batteries used to ensure the battery remains within its ideal working parameters, as well as load balancing to ensure all cells complete

charging at the same time, thus preventing damage through overcharging [26].



Fig 1.27 18650 Lithium Battery

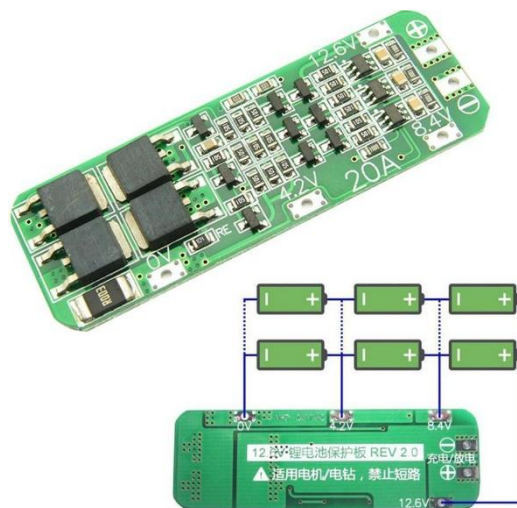


Fig 1.28 18650 Charger BMS Protection Board

1.3.17 Analog Joystick

An analog joystick shown in Fig1.29, sometimes called a control stick joystick or thumb stick is an input device for a controller that is used for two-dimensional input. An analog joystick is similar to two potentiometers one for the vertical movement(Y-axis) and other for the

horizontal movement (X- axis). The joystick also comes with a select switch. It can be very handy for retro gaming, robot control or RC cars [27].

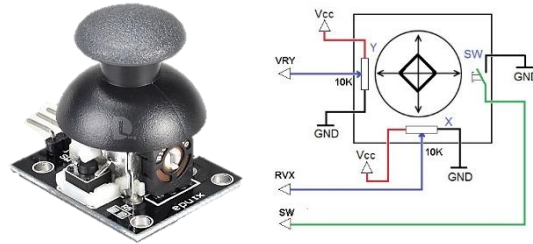


Fig 1.29 Analog Joystick

1.3.18 ESP8266 Wi-Fi Module

The module Shown in Fig1.30 is best suited for Internet of Things (IoT) at its low cost, low power consumption capability as it requires 3.3V power, built in Wi-Fi module, integrated TCP/IP protocol stack, easy to flash and erase firmware and is usb powered. As the IoT application module it can deploy in home automations, home appliances, industrial wireless network, sensor networks fields [28]. ESP8266 Wi-Fi Module Specifications shown in table 1.8.

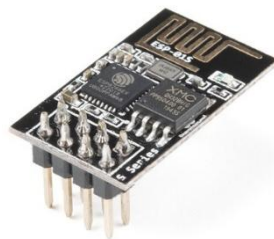


Fig 1.30 ESP8266 WIFI Module

Microcontroller	ESP8266
Memory	32 bit
Processor	TenSilica L 106
Processor Clock	80MHz-160MHz
RAM	36Kb
Storage	16 Mb
Built-in WiFi	2.4GHz supports 802.11 b/g/n
Operating Voltage	3.0V 3.6V
Operating Current	80mA(Average)
Operating Temperature Range	-40oC - 125 oC

Table 1.8 ESP8266 Wi-Fi Module Specifications

1.4 Design layout and Implementation

In this design, a forklift car has a four wheels drive system. The left two wheels are driven by only one signal. The right two wheels are also driven by only one signal from the controller.

1.4.1 Robot Frame

Different agricultural environments and applications may require robots of different widths, heights and lengths, or different frame stiffness and lifting capabilities. Even farms that grow the same kind of crop may require different robots due to dissimilarities in existing infrastructure.

To accommodate the variation of needs that may occur, it is important to include a great deal of customizability in the frame design. This is achieved by constructing the robot's frame from aluminum tubes that are held together by specially-designed clamps. Clamps and brackets are also used to connect modules to the frame and, in some cases, to each other. Relevant modules share the same mechanical interface, e.g., a steering

module may connect to the robot's frame via a suspension module, or it may just as easily be connected directly to the frame in cases where a suspension module is not needed.

Creating a robot with custom geometry is a matter of cutting aluminum tubes to the correct lengths and clamping them together. The robot's frame may easily be made stiffer by clamping extra members to the frame, or more flexible by omitting frame members. A stiffer frame may carry more load, whilst a more flexible frame will increase the robot's ability to keep wheels in connection with the ground. This means that completely custom frames may be achieved by simple means, also the ones that are not based on the above-mentioned clamp and tube system [29].

1.4.2 Suspension Module

A suspension module shown in Fig1.31 can be connected between the robot's frame and a steering module, or to a bracket on a drive module in the case of a differential drive robot. The module is optional, but will increase the robot's ability to keep the wheel in contact with the ground. It will also help with absorbing shocks.

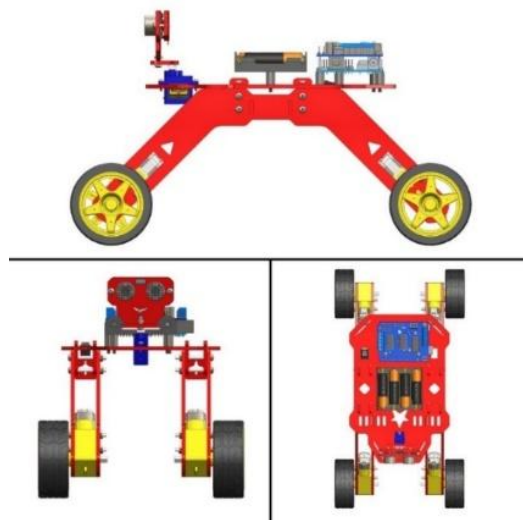


Fig 1.31 Robot Suspension Module

1.4.3 Modules for Sensors

Plastic Storage Container Boxes of different sizes and shapes were used to put the sensors inside and protect them from various environmental conditions such as rain and dust. These plastic boxes are characterized by their lightweight so as not to hinder the movement of the vehicle or reduce its speed. It is also characterized by high durability to withstand the various environmental conditions in which the robot moves, as the robots that are present in rugged agricultural environments are exposed to many dangers such as rocks or falling into holes, and these accidents may affect the sensors and cause them to malfunction, so they must be protected in storage boxes that are durable. Only parts of this sensor that are specialized in the measurement process should be shown. The Plastic Storage Containers Boxes shown in Fig1.32.



Fig 1.32 Plastic Storage Containers Boxes

1.4.4 Electric System

The robot is powered by 7800mAh or 2200mAh, 12 VDC lithium-ion battery packs. The battery management system (BMS) of each battery connects to the robot's, which means that parameters status may be obtained. The battery connects to the motor driver so supplying the motor driver with 12v DC, then the Motor Driver supplies the microcontroller with 9V and also feeds the wheels with electrical energy as well, and the Arduino feeds the

sensors with power, and an external power supply is connected to supply the rest of the sensors with sufficient electrical energy. Various robots may have different numbers of motor controllers and sensors, which all need power and a means of communication. It has been a key priority to simplify electric interfaces between modules. To this end, connectors and cables for connecting modules have been standardized, and a set of circuit boards has been designed, most important being the previously-mentioned main battery enclosure board.

1.4.5 Kinematic Model

The geometry of the robot is presented in Fig1.33. To consider the model of the four-wheel skid steering mobile robot (SSMR) it is assumed that the robot is placed on a plane surface where (X_I, Y_I) is the inertial reference frame and (X, Y) is a local coordinate frame fixed on the robot at its center of mass (COM). The position of the COM is (x, y) with respect to the inertial frame and ϑ is the orientation of the local coordinate frame with respect to the inertial frame [30].

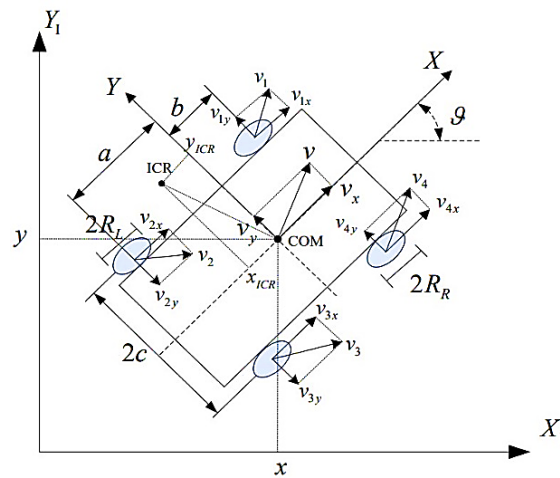


Fig 1.33 Mobile Robot Geometry

As depicted in Fig.33, a is the distance between the center of mass and the front wheels axle along X , b is the distance between the center of mass and the rear wheels axle along X , c is half distance between wheels along Y and R_L, R_R are the radii of left and right wheels respectively. The coordinates of the instantaneous center of rotation (ICR) are (x_{ICR}, y_{ICR}) . Assuming that the robot moves on a horizontal plane the linear velocity with respect to the local frame is given by

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ 0 \end{bmatrix}$$

and its angular velocity is given by

$$\boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \omega \end{bmatrix}$$

The state vector with respect to the inertial frame is

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \vartheta \end{bmatrix}$$

The time derivatives of (3) denotes the robot's velocity vector and is given by

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\vartheta} \end{bmatrix} = \begin{bmatrix} \cos \vartheta & -\sin \vartheta & 0 \\ \sin \vartheta & \cos \vartheta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

Assuming that longitudinal slip between the wheels and the surface can be neglected we have the following equation,

$$v_{ix} = R_i \omega_i$$

where v_{ix} is the longitudinal component of the total velocity vector v_i of the i -th wheel expressed with respect to the local frame and R_i is the rolling radius of that wheel. If we take into account all wheels (Fig. 33), the following relationships between the wheels can be obtained,

$$v_L = v_{1x} = v_{2x}$$

$$v_R = v_{3x} = v_{4x}$$

$$v_F = v_{1y} = v_{4y}$$

$$v_B = v_{2y} = v_{3y}$$

where v_L refers to the longitudinal coordinates of the left wheel's velocities, v_R refers to the longitudinal coordinates of the right wheels velocities, v_F refers to the lateral coordinates of the front wheels velocities and v_B refers to the lateral coordinates of the rear wheels velocities. Unlike other mobile robots, lateral velocities of the four-wheel skid steering mobile robot are generally nonzero since from its mechanical structure the lateral skidding is necessary if the robot changes its orientation. Therefore, in order to complete the kinematic model, the following nonholonomic constraints in Pfaffian form is introduced

$$\begin{bmatrix} -\sin \vartheta & \cos \vartheta & -x_{ICR} \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\vartheta} \end{bmatrix} = A(q)\dot{q} = 0$$

Then we have

$$\dot{q} = S(q)\eta$$

Where

$$s(q) = \begin{bmatrix} \cos \vartheta & x_{ICR} \sin \vartheta \\ \sin \vartheta & -x_{ICR} \cos \vartheta \\ 0 & 1 \end{bmatrix}$$

$$\eta = \begin{bmatrix} v_x \\ \omega \end{bmatrix}$$

$S(q)$ is a full rank matrix, whose columns are in the null space of $A(q)$,

$$S^T(q)A^T(q) = 0$$

It is noted that since $\dim(\eta) = 2 < \dim(q) = 3$, describes the kinematic of a sub-actuated robot with the nonholonomic constraint given above.

In this work we are only interested in abrupt faults which occur in the actuators of the mobile robot and as consequence, we make the following assumptions

- Assumption 1: When the mobile robot starts functioning all its components are in normal mode.
- Assumption 2: The magnitude of the noise is assumed to be significantly smaller than the magnitude of the faults.
- Assumption 3: Regarding the wheel radius the following inequalities are satisfied:

$$R_R + \delta R_R > 0 \quad \& \quad R_L + \delta R_L > 0$$

According to this assumption, faults that result in the complete loss of the wheel are not considered [30].

1.5 Software Design Layout

As for the software side, which is no less important than the practical side and connecting electronic parts, we have used different languages in programming and designing the platform in which the reading data is sensed by the sensors. The platform was designed to be available on the Windows operating system and we made a special application to control the robot via Bluetooth and the application is available for the Android system. In this chapter, we will provide a brief overview of the languages and applications that are used in design and programming.

1.5.1 Arduino IDE Software

the Arduino IDE (Integrated Development Environment). Built up with the 8-bit Atmel AVR microcontroller's that are manufactured by Atmel or a 32-bit Atmel ARM, these microcontrollers can be programmed easily using the C or C++ language in the Arduino IDE. The program code written for Arduino is known as a sketch. The software used for developing such sketches for an Arduino is commonly known as the Arduino IDE [31]. Arduino IDE was used, in this project the code was written in C++, then the code was burned on the board. Fig1.34 shows the Arduino IDE Software program.

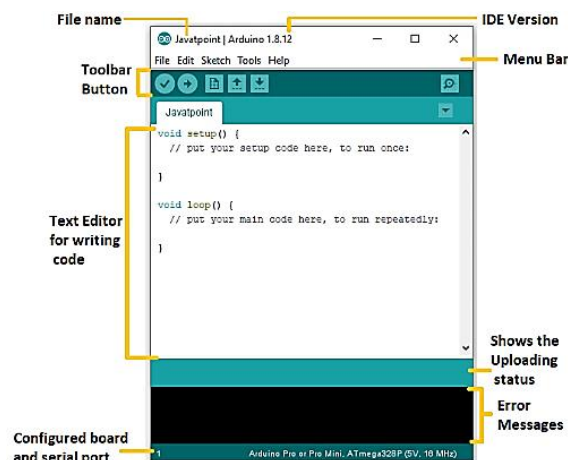


Fig 1.34 Arduino IDE Software program

1.5.2 Visual Studio

Visual Studio is one of the most famous programs that contains many different programming languages and libraries that are used in design and programming, including the C# language, as we designed the main interface of the platform which display the readings that are received from the sensors on the robot by using the C# language by using the Visual Studio to be compatible with the Windows operating system.

1.5.3 MIT App Inventor

The MIT App Inventor user interface includes two main editors: the design editor and the blocks editor. The design editor, or designer as shown Fig1.35, is a drag and drop interface to lay out the elements of the application's user interface (UI). The blocks editor as shown Fig1.36 is an environment in which app inventors can visually lay out the logic of their apps using color-coded blocks that snap together like puzzle pieces to describe the program. To aid in development and testing, App Inventor provides a mobile app called the App Inventor Companion (or just "the Companion") that developers can use to test and adjust the behavior of their apps in real time. In this way, anyone can quickly build a mobile app and immediately begin to iterate and test [32]. We designed the application for controlling the movement of the robot using the MIT App Inventor because it is easy to program and fast and does not require any complexity as in the rest of the other languages as it supports many designs and it support Bluetooth and Wi-Fi so you can make any application on mobile phone to connect with other devices that support Bluetooth or Wi-Fi. The application can also be tested before converting it to Android.



Fig 1.35 MIT App Inventor Design Editor



Fig 1.36 MIT App Blocks Editor

1.5.4 X-CTU Software

X-CTU is a Windows-based application provided by Digi shown in Fig1.37. This program was designed to interact with the firmware files found on Digi's RF products and to provide a simple-to-use graphical user interface to them. When launched, four tabs will be seen across the top of the program [33].

- 1- **PC Settings:** Allows a customer to select the desired COM port and configure that port to fit the radios settings.
- 2- **Range Test:** Allows a customer to perform a range test between two radios.

- 3- **Terminal:** Allows access to the computers COM port with a terminal emulation program. This tab also allows the ability to access the radios' firmware using AT commands.
- 4- **Modem Configuration:** Allows the ability to program the radios' firmware settings via a graphical interface. This tab also allows customers the ability to change firmware versions



Fig 1.37 X-CTU software

Chapter Two

Practical

Implementation

Chapter Two Practical Implementation

2.1 The Aim of Chapter

In this chapter, we will discuss the practical side in detail for hardware and software, Where the practical work was divided into several sections in terms of design as well as programming in order to facilitate the matter when everything is done at the same time, also we will discuss the simulator that was used to simulate the electronic circuit of the device.

2.2 Hardware Design flowchart

The Figure 2.1 Shows the Hardware design flowchart

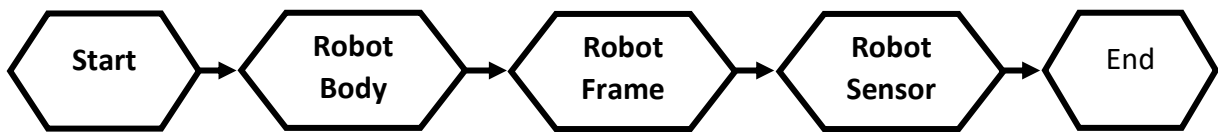


Fig 2.1 Hardware Design Flowchart

2.2.1 Robot Body Design

In the beginning, we faced many challenges in terms of designing the body of the rover because, as we know, agricultural environments contain many variables and many daily challenges, for example the temperature difference or the change in humidity, as well as we do not forget the roughness of the soil, and all these factors greatly affect the vehicle. Therefore, it was necessary to make a solid body for the vehicle so that it can withstand harsh conditions, and at the same time it should not hinder its movement within the field, that is, it should be flexible and light in movement, so we have chosen a metal base made of hollow light steel as

shown in Fig2.2. With 18.5 cm length, 11 cm width, and 5 cm high, as this base represents the main support for the body of the vehicle and the other parts and components it contains, and it was chosen with these measurements so as not to constitute excessive weight on the vehicle and thus hinder its movement.



Fig 2.2 Metal Plate

As for the upper structure of the vehicle, it was obtained from the internal parts of the desktop, in which the hard disk is placed, where these pieces were connected to each other and fixed by screws to form the upper structure of the vehicle as shown in the Fig.2.3.



Fig 2.3 Hard Disk Drive Bo

2.2.2 Robot Frame

To manufacture the pieces on which the tires are based, we brought PVC pipe plates, where we designed the shape of the base of the frame, then we cut it to become the shape of the design then we put the motors on it using screws, and tires, and the shape 2.5 represents the final design for it.



Fig 2.4 Robot Frame and Tires

2.2.3 Robot Sensors

We placed the sensors in different places of the robot according to the type of the sensor, for example the CO2 sensor placed at a height of 40 cm on the robot. The same applies to the temperature and humidity sensor. As for the GPS and ESP8266, they were put directly on the vehicle because they do not need a specific distance for measurement. As for the soil moisture sensor, it was installed at the base of the vehicle to be close to the soil to take measurements when needed. The compass was installed at the head of the robot so that it performs the calibration process correctly every time measurements are taken. ZigBee has been installed directly on the Arduino Mega using the shield. Then we installed the rain sensor in a single open place so that it could sense the rain without any obstacles, and the water pump was installed at the end of the robot.

2.3 Circuit Design

About Arduino and electronic parts, we have carried out two important stages. First, we made a diagram of the electrical circuit of the robot, and secondly, we designed a diagram of the electrical circuit of the control unit (Zigbee and Arduino).

2.3.1 Main Circuit Diagram

Fig 2.5 represents the main diagram of the vehicle's electrical circuit and how to connect the sensors with the Arduino. We note that we used the Arduino Mega 2560 because it contains a large number of pins that we need because of the many sensors that we have. It also contains four serial ports, and this is one of the important features of the Arduino Mega 2560, which helped us a lot in This design is because, as we know, the majority of electronic parts and sensors communicate with the Arduino serially, and we can read the results by using the serial monitor in the Arduino IDE, which we got acquainted with in the Chapter one. The table 2.1 represents the connection map of the main circuit.

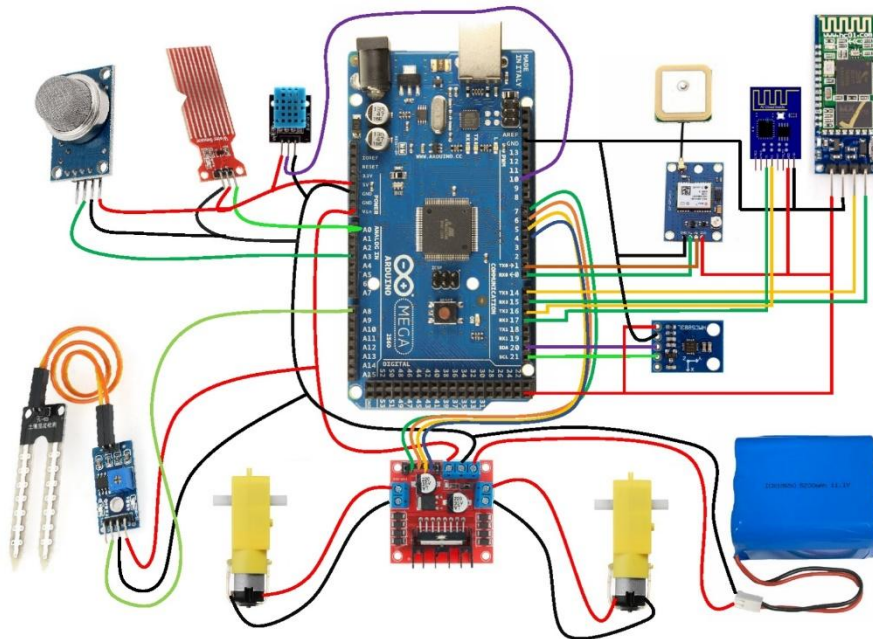


Fig 2.5 Main Circuit Diagram

2.3.2 Circuit Simulation

A simulation of the electrical circuit diagram was made using the Proteus simulation program before applying the circuit in practice, as shown in Fig 2.6.

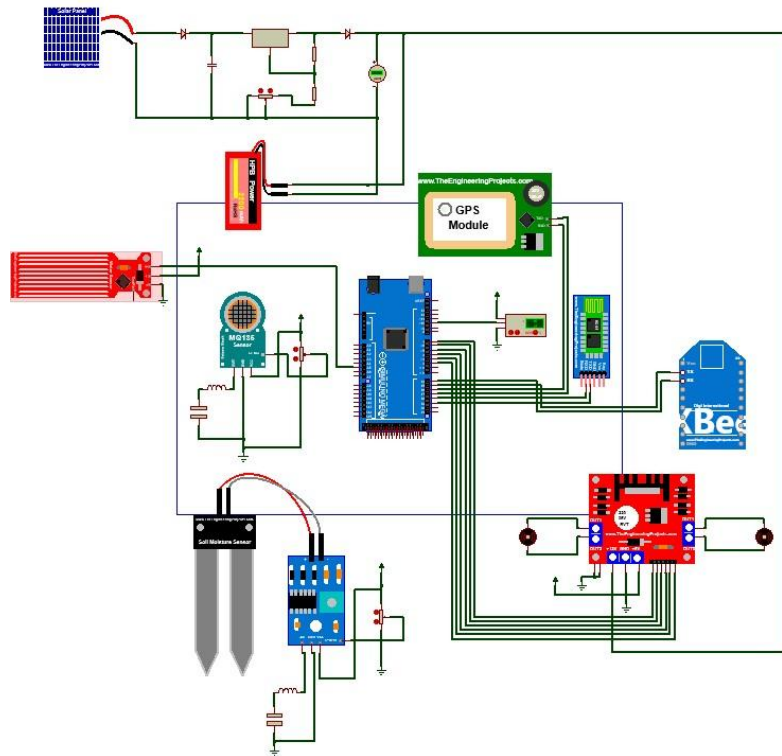


Fig 2.6 Circuit Simulation Diagram

2.3.3 Control Unit Circuit Diagram

A control unit for the movement of the vehicle was also designed by using the XBEE with Arduino Uno and the joystick. The circuit diagram was connected and a simulation was also made using the Proteus simulation program. Similar results were obtained for the practical side, as we were able to control the vehicle for a distance of 1000m using the XBEE (Zigbee). The control unit Simulation Circuit is shown in Fig 2.7 and Fig 2.8 shows the control unit of the robot.

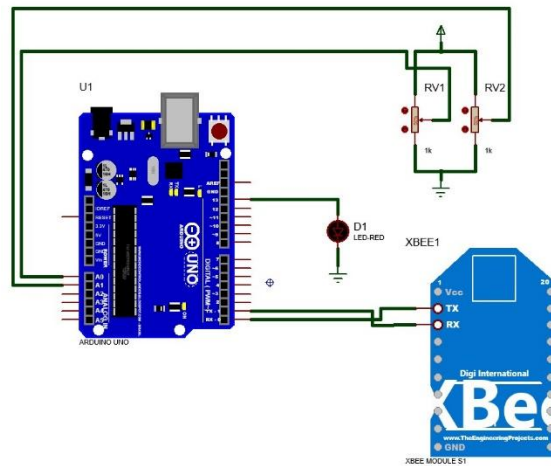


Fig 2.7 Control Unit Circuit Simulation Diagram

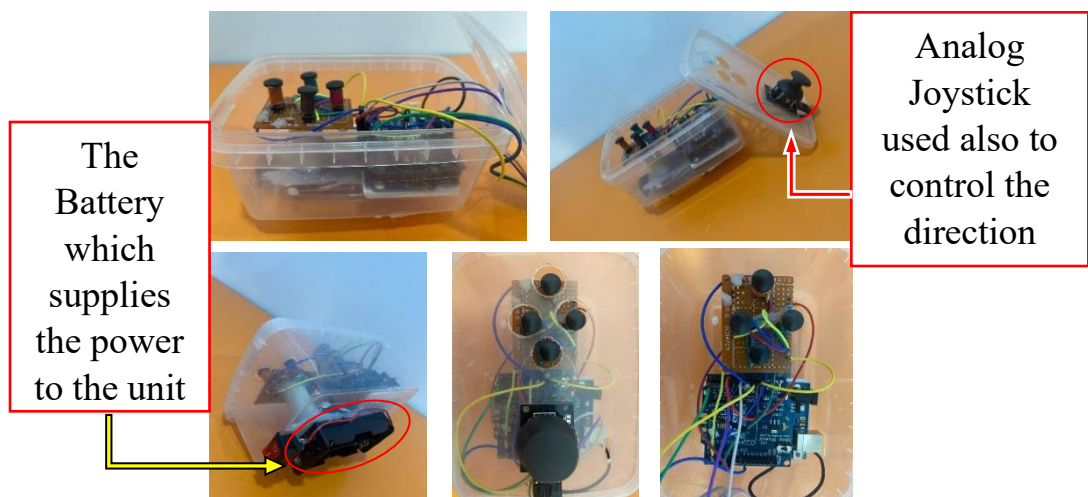
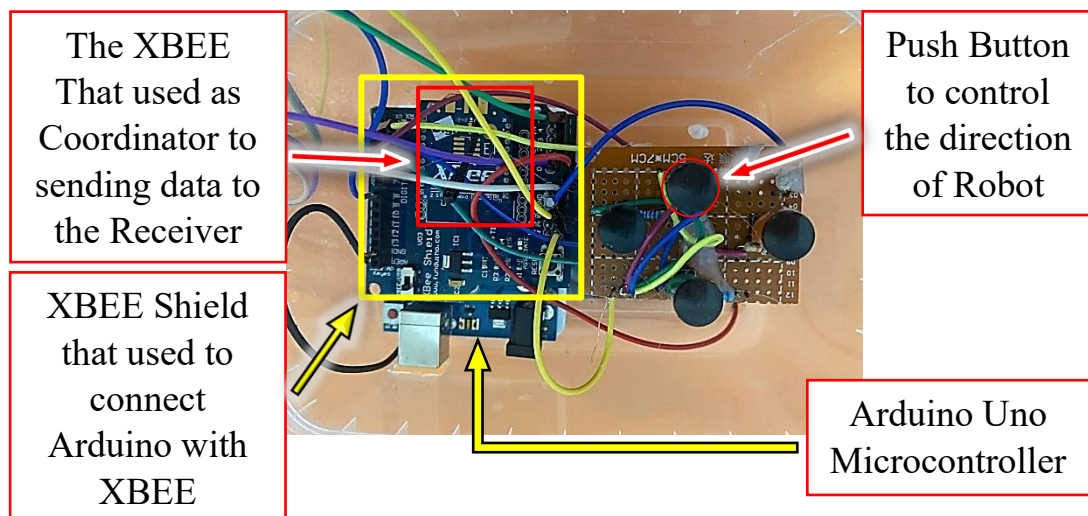


Fig 2.8 Control Unit of The Robot

2.3.4 Actually Design

After getting acquainted with all the methods of connecting all the electronic parts and the external design, Fig 2.9 has some pictures of the robot that we designed. These pictures are all real for the vehicle. They were collected from the beginning of the work until reaching the end of the project.



Fig 2.9 Real Pictures of the robot

2.4 Software Design

As we mentioned in the chapter 1, more than one programming language and more than one program were used to do the programming of the robot. We will mention in this chapter the details of the software programming process.

2.4.1 Arduino IDE Algorithm

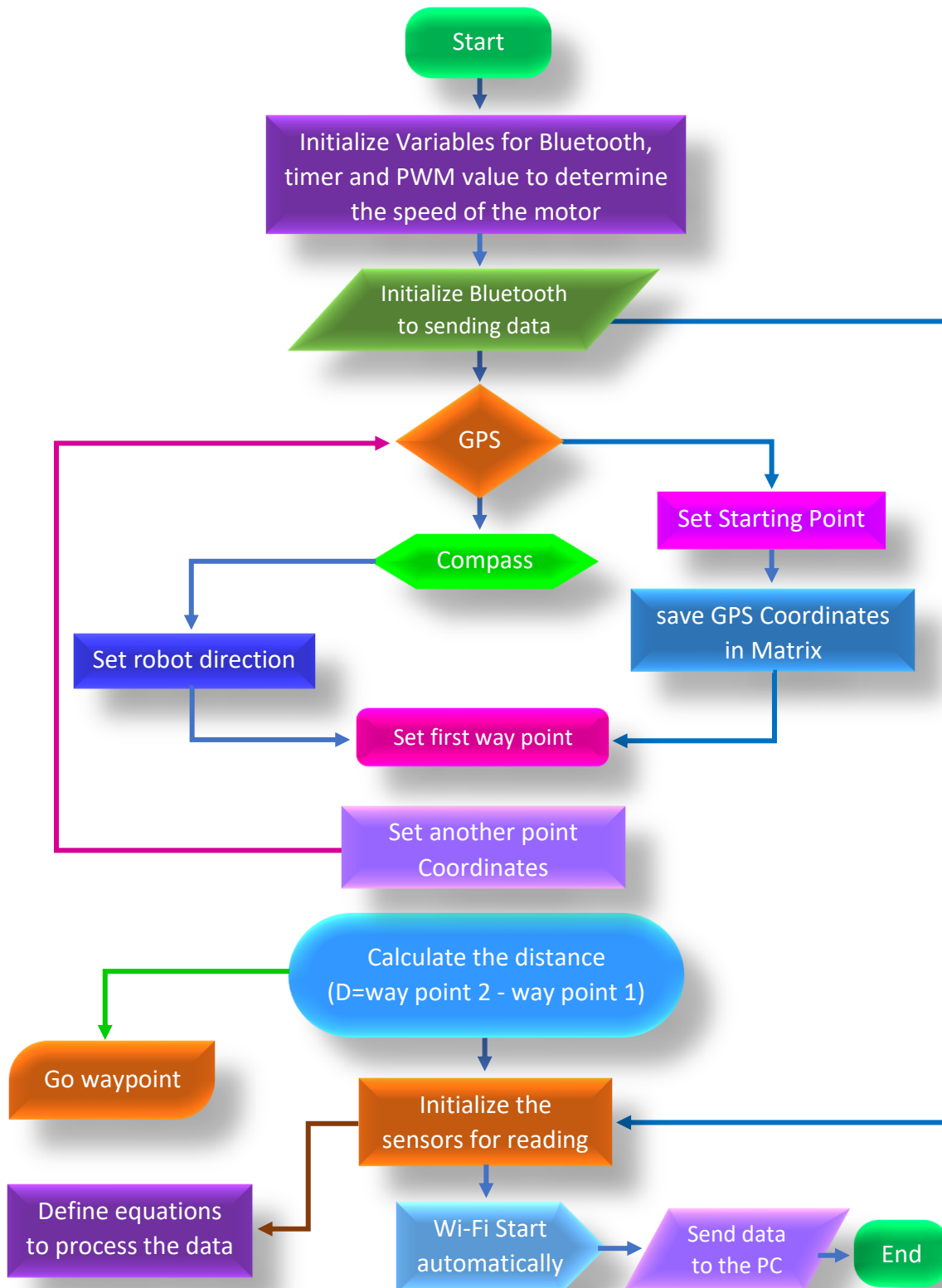


Fig 2.10 Arduino IDE Algorithm Flowchart

2.4.2 MIT App Inventor Algorithm

We designed the Android app algorithm as shown in Fig 2.12 based on the flowchart as shown in Fig 2.11.

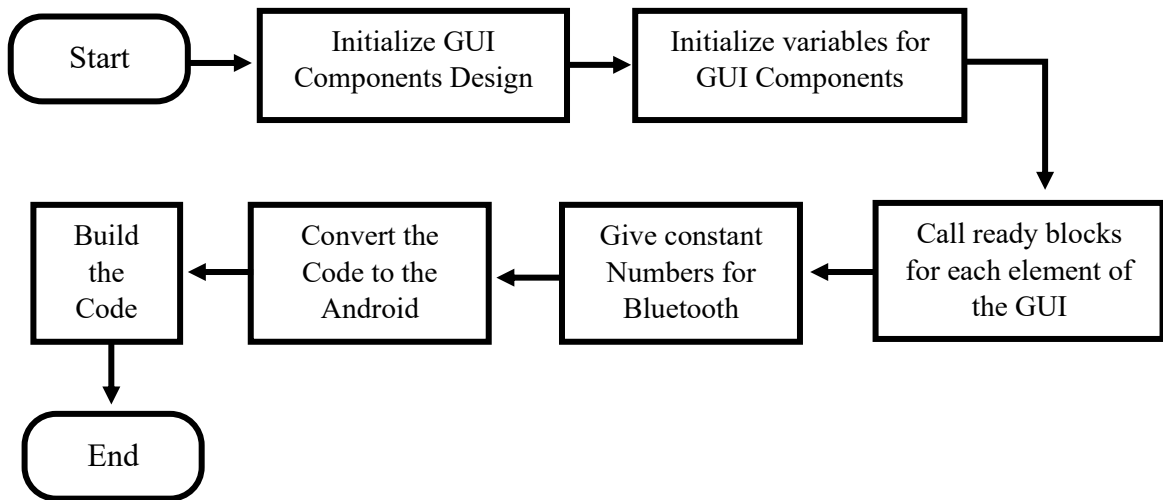


Fig 2.11 MIT App Inventor Flowchart



Fig 2.12 MIT App Inventor Application Design

The Details of the Application are Described below:

- 1- Connect BT:** When press this button in make a Bluetooth connection between the mobile phone and Bluetooth module.
- 2- Disconnect BT:** Cancel the connection between mobile and Bluetooth module.
- 3- GPS Info:** get the robot position information (longitude and latitude) by using GPS module.
- 4- Set Way:** set waypoint coordinates and calculate the distance to make waypoint from home to destination.
- 5- Done:** Complete the waypoint and make the robot ready to move.
- 6- Clear All:** Clear all data GPS, Compass and waypoints to set new data.
- 7- Forward:** Make the robot move forward
- 8- Reverse:** Make the robot move in Reverse.
- 9- Left:** Turn the robot in the left side.
- 10- Right:** Turn the robot in the right side.
- 11- Stop:** Stop the robot (stop all motors).
- 12- Left 90:** Turn the robot 90 degrees in the Left side.
- 13- Right 90:** Turn the robot 90 degrees in the Right side.
- 14- Turn Around:** Make the robot rotate in 360 degrees.
- 15- Set Heading:** set the robot head direction by using the compass.
- 16- Calibrate:** Calibrate the compass before moving to initialize the robot direction.
- 17- Compass drive:** Make the compass driving the robot without GPS.
- 18- Ping on/off:** Make the Ultrasonic sensor working to avoid a collision with obstacles.
- 19- Send:** It is used to send a specific speed to the robot by using the PWM

2.4.3 GUI Platform for Windows Application Algorithm

We designed the platform algorithm through which the sensor data is displayed on the computer using the C# language, as shown in Fig 2.13.

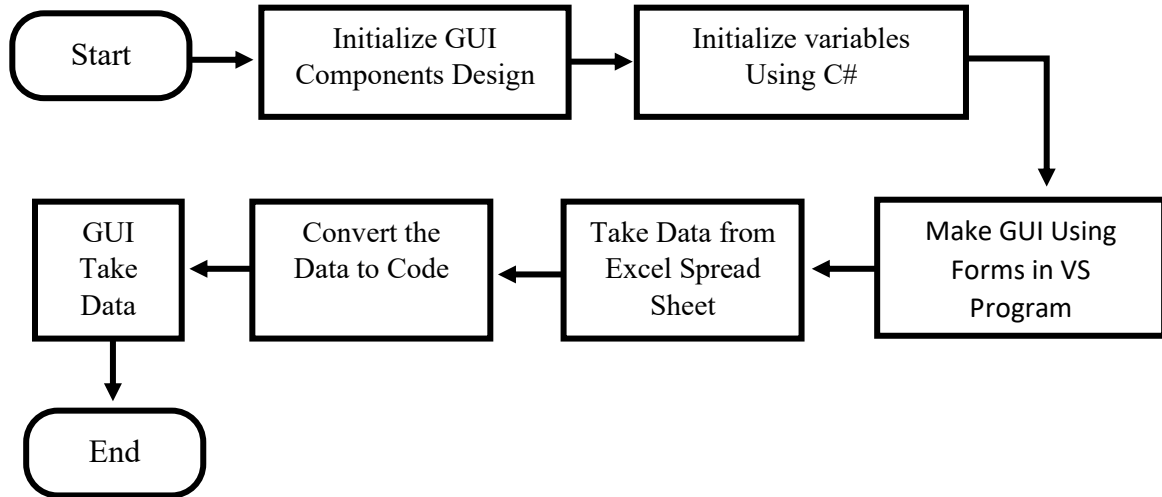


Fig 2.13 GUI Platform Algorithm Flowchart

Chapter Three

Project results

Chapter Three Project Results

3.1 Aim Of Chapter

In this chapter, which is the last chapter, we will present the results that we have obtained when conducting practical experiments on the robot in different environments and under different conditions. This chapter will deal with several aspects of the experiment, including the mechanical aspects of the project, the communication accuracy between sender and receiver, also the results of the data read by the sensors, their accuracy, and how they are stored and archived.

3.2. Field Trials

As stated in Chapter 2, the standard configuration of the robot has a strong metal base, four frame modules and two suspension modules. This makes the robot agile and powerful, but with four motor-gear assemblies, it is naturally more costly to make than a differential drive robot with only two motor-gear assemblies. It is important to understand this trade-off between performance and cost when it comes to deciding on a specific robot configuration for a given task. To quantify the differences between configurations in terms of performance, a series of tests was conducted. The configurations that are studied are all configurations intended for outdoor use with frames made from standard frame components. The standard robot is fitted with one 7400mAh battery pack in the main battery enclosure. The tests are described in more detail below.

3.2.1 Incline Test

In this test, each robot starts from a standstill on level ground and accelerates to a speed of 1.0 m/s. The robot then enters a 30-degree steep,

grass-covered incline and maintains constant motor command until it reaches the top. After reaching the top, the robot starts reversing, descends to the middle of the incline, comes to a stop, starts from a standstill and climbs to the top one more time. The test finishes when the robot reaches the top for the second time, or fails to climb the incline. The robots are teleoperated for the duration of the test, and all runs are recorded by a static camera. The incline is

depicted in Fig.3.1.



Fig.3.1. The incline Test in The Robot

3.2.2 Passing over Obstacles

In this test, each robot starts from a standstill and accelerates to a speed of 0.25 m/s, which is maintained for the duration of the test. The robot is teleoperated through a straight course populated by several obstacles, which the robots must overcome. The obstacles are numbered from start to finish, with dimensions. The obstacles are placed as shown in Fig 3.2.



Fig.3.2. The obstacles Test in The Robot

3.3. Results

3.3.1 Incline Test

All four-wheel drive configurations, that is Configurations 1, 2 and 3, succeeded in climbing the incline from a flying start, as well as from standstill mid-incline. These configurations had no problems while descending and stopping in the middle of the incline. With the drive wheels in the front, Configurations 4 and 5 failed at reaching the top as they both lost traction in the middle of the incline. When driving with drive modules at the rear, however, both Configurations 4 and 5 were able to reach the top, from a flying start, descending and starting from rest mid-incline, despite losing traction sporadically.

3.3.2 Passing over Obstacles

All four-wheel drive configurations, Configurations 1, 2 and 3, passed all obstacles. Configuration 4 failed at Obstacle 4, while Configuration 5

failed at the very first obstacle. Obstacles 1 and 2 were rotated 90 degrees (in the horizontal plane) for Configurations 4 and 5 to avoid obstacles getting stuck between the robot's front and rear wheels. The only configuration that managed to keep all wheels in connection with the ground throughout the test was Configuration 3, the robot with four-wheel drive and open frame. Configuration 1 lost connection with the ground on one wheel when driving over two obstacles simultaneously on diagonally-opposite corners. It also lifted slightly on one wheel when passing Obstacle 5. Configuration 2 lost connection with the ground on one or two wheels on all the obstacles. Fig.3.3 shows the robot during the test.



Fig.3.3. The Robot During the Test

3.3.3 Field Trials Discussion

Unsurprisingly, four-wheel drive performed better than two-wheel drive in all the presented field trials. Other than having two less propulsion motors, there seems to be two important reasons why the differential robots fail. One, the diameter of the rear caster wheels is too small for the obstacles. As almost all obstacles were taller than half the diameter of the caster wheel,

the robot struggled to pull the passive wheels over the obstacles and ended up either losing traction on the drive wheels or pulling the obstacles along the ground. This was especially apparent when comparing the closed and open frame configurations. When the front drive wheel of the closed frame configuration climbed an obstacle, the rear wheel was lifted from the ground because of the stiffer closed frame. This helped the caster clear the obstacle. With the open frame, the caster remained in connection with the ground until it hit the obstacle, effectively stopping the robot.

The second reason why the differential robots underperformed in these tests is that the robot is too light-weight to maintain good traction, especially on the side with no battery. When driving the robot around the test area, the robot performed much better when carrying the 137-kg load compared to driving with no extra load. Similarly, the differential drive robots were able to get up the incline when the drive modules were at the aft, supporting more of the robot's weight, but not when the drive modules were at the front. This suggests that these configurations may need some added weight when operating in difficult terrain.

3.4 Power Saving Trial

We note that when connecting a battery with 7400 mAh, the experience of driving the robot is much better than using a 2400 mAh battery. It should also be noted that despite the use of a relatively cheap battery, the battery power saving was very good, especially in soil that does not contain obstacles. A lot, so the torque of the motors decreases and is not spent much power. Also, as a result of putting the Zigbee in the sleep mode when no data is sent, this also reduces battery savings and prevents it from running out

very quickly. Also, the process of sending sensor data was done every second at the beginning, then the time was increased to Half an hour between one reading and another, and this also saved battery power.

3.5 Data measurement accuracy

As for the accuracy of the data measurement, we will display the most sensors reading data that are used in this project.

3.5.1 GPS reading

We noticed when trying the robot in different places that the GPS reading changed every time depending on the place. When trying the robot inside the house, it was very difficult to obtain GPS coordinates (latitude and longitude), on the contrary, in open places, a satellite connection was made in Less than 30 seconds and accurate coordinates of the place were given, and even more than one satellite was contacted, as the number of satellites reached 5 in one of the field experiments, and since most agricultural fields are open only containing vegetation, this GPS is very suitable for such Places where the measurement rate is not less than 6 meters, and this distance is very appropriate, as there is certainly no field with these small measurements, and it updates the data every second as long as there is a connection to the satellite and it is low cost and saving the power. Fig 3.4 shows the GPS obtained data using Arduino serial monitor.

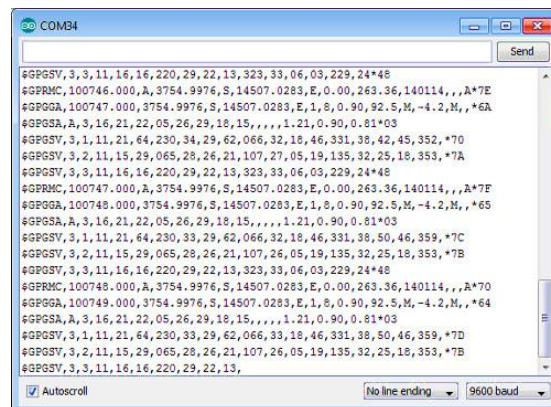


Fig 3.4 GPS obtained data using Arduino serial monitor

3.5.2 Compass reading

The compass had the main role in this project by determining the coordinates of the place for the robot to determine the correct direction for the destination. Therefore, at the beginning, when using it for the first time, it must be calibrated using an external program as shown in Fig 3.5 for two minutes until it determines the directions and angles correctly, then it put to the robot, as we noticed that there were There is a very small percentage of error in the readings when tested indoors, and this is due to the presence of different minerals that interfere with their work and their magnetic field, but when tested in an open outdoor place away from minerals, these readings were improved and more accurate data obtained.

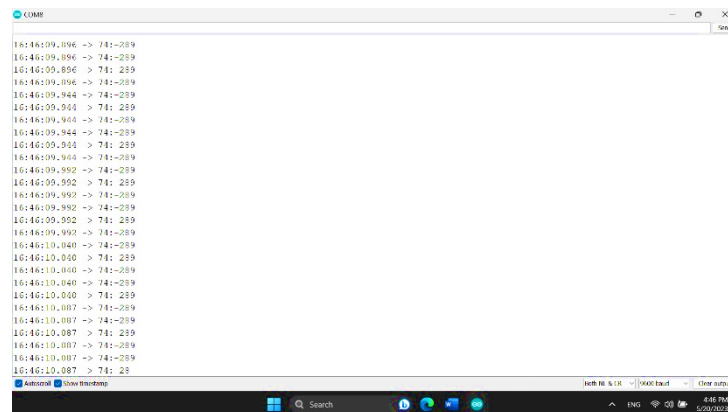


Fig 3.5 Calibrating the compass using Arduino

3.5.3 Zigbee Data Communication

Although the transmission of data through Zigbee is relatively slow, but it has achieved the best results in controlling the movement of the robot, because it can transmit data to a fairly good distance, as the signal was delivered to approximately 1000 meters outdoor and to approximately 45 meters indoor ,and it does not use very high power, unlike other RF modules

that consume the battery capacity quickly and reduce its life, due to the sleep mode feature, which is one of the advantages of the Zigbee, and that is why it was chosen in this project, and its price is relatively cheap.

3.5.4 Sensor reading accuracy

Despite the use of relatively inexpensive sensors, they have given the best results in terms of accuracy in reading data, as a result of connecting them correctly and adding some other electronic elements that can increase the accuracy of their measurement relatively, as well as not forgetting the process of programming them in terms of choosing the appropriate libraries. It is represented by the simplest software code, so good results were obtained for the data sensing process, and the Fig3.6 below shows the obtained results, as they are sent to an excel file using Wi-Fi, then we store and archive them according to date and time, and we note that the measurement of the CO2 percentage was initially relatively high, because the MQ-135 gas sensor contains a capacitor inside it. When starting work, this capacitor heats up for about 5 minutes, then the measurement begins accurately, and it can also be calibrated through the potentiometer that is put in the PCB circuit of the sensor. Fig.3.7 shows the MQ-135 and DHT-11 reading data and Fig.3.8 Shows the rain sensor and soil moisture sensor reading data.

	A	B	C	D	E	F	G
1	Temperature	soil moisture	CO2 Level (PPM)	Rain	Date	Time	
2	20	9%	755	0%	3/12/2022	7:12PM	
3	21	9%	755	0%	3/12/2022	7:13PM	
4	20	12%	623	21%	3/12/2022	7:14PM	
5	20	10%	621	23%	3/12/2022	7:15PM	
6	20	12%	655	16%	3/12/2022	7:16PM	
7	20	12%	622	27%	3/12/2022	7:17PM	
8	19	13%	545	28%	3/12/2022	7:18PM	
9	20	13%	548	57%	3/12/2022	7:19PM	
10	21	14%	602	49%	3/12/2022	7:20PM	
11	20	15%	633	55%	3/12/2022	7:21PM	
12	21	16%	564	61%	3/12/2022	7:22PM	
13	20	16%	562	67%	3/12/2022	7:23PM	
14	20	16%	425	70%	3/12/2022	7:24PM	
15	19	17%	467	79%	3/12/2022	7:25PM	
16	20	18%	464	85%	3/12/2022	7:26PM	
17	19	18%	434	86%	3/12/2022	7:27PM	
18	20	19%	423	83%	3/12/2022	7:28PM	

Fig3.6 The Data Results That Obtained

```

17:45:01.893 -> CO2 Level int: 750      Temperature: 28      Humidity: 35%
17:45:01.939 -> CO2 Level int: 731      Temperature: 27      Humidity: 35%
17:45:02.030 -> CO2 Level int: 761      Temperature: 29      Humidity: 35%
17:45:02.083 -> CO2 Level int: 654      Temperature: 29      Humidity: 33%
17:45:02.131 -> CO2 Level int: 596      Temperature: 27      Humidity: 35%
17:45:02.235 -> CO2 Level int: 623      Temperature: 26      Humidity: 36%
17:45:02.273 -> CO2 Level int: 512      Temperature: 28      Humidity: 35%
17:45:02.368 -> CO2 Level int: 518      Temperature: 27      Humidity: 33%
17:45:02.415 -> CO2 Level int: 620      Temperature: 28      Humidity: 35%
17:45:02.464 -> CO2 Level int: 723      Temperature: 28      Humidity: 36%
17:45:02.560 -> CO2 Level int: 721      Temperature: 28      Humidity: 35%
17:45:02.609 -> CO2 Level int: 604      Temperature: 29      Humidity: 35%
17:45:02.657 -> CO2 Level int: 632      Temperature: 28      Humidity: 34%
17:45:02.751 -> CO2 Level int: 513      Temperature: 27      Humidity: 35%
17:45:02.797 -> CO2 Level int: 621      Temperature: 28      Humidity: 35%
17:45:02.844 -> CO2 Level int: 543      Temperature: 28      Humidity: 37%
17:45:02.890 -> CO2 Level int: 526      Temperature: 26      Humidity: 35%
17:45:02.986 -> CO2 Level int: 517      Temperature: 28      Humidity: 35%
17:45:03.033 -> CO2 Level int: 497      Temperature: 27      Humidity: 35%
17:45:03.126 -> CO2 Level int: 481      Temperature: 28      Humidity: 36%
17:45:03.173 -> CO2 Level int: 492      Temperature: 28      Humidity: 36%
17:45:03.268 -> CO2 Level int: 466      Temperature: 28      Humidity: 36%
17:45:03.350 -> CO2 Level int: 432      Temperature: 28      Humidity: 36%
17:45:03.383 -> CO2 Level int: 474      Temperature: 28      Humidity: 36%
17:45:03.464 -> CO2 Level int: 429      Temperature: 28      Humidity: 35%
17:45:03.549 -> CO2 Level int: 475      Temperature: 29      Humidity: 34%
17:45:03.584 -> CO2 Level int: 501      Temperature: 28      Humidity: 35%
17:45:03.633 -> CO2 Level int: 585      Temperature: 28      Humidity: 35%
  
```

Fig3.7 MQ-135 and DHT-11 reading data

```

17:51:28.795 -> Rain sensor: 08      Soil Sensor: 9%
17:51:28.795 -> Rain sensor: 08      Soil Sensor: 10%
17:51:28.842 -> Rain sensor: 08      Soil Sensor: 10%
17:51:28.890 -> Rain sensor: 08      Soil Sensor: 10%
17:51:28.934 -> Rain sensor: 28      Soil Sensor: 10%
17:51:28.972 -> Rain sensor: 28      Soil Sensor: 10%
17:51:29.019 -> Rain sensor: 18      Soil Sensor: 10%
17:51:29.066 -> Rain sensor: 08      Soil Sensor: 9%
17:51:29.113 -> Rain sensor: 08      Soil Sensor: 9%
17:51:29.160 -> Rain sensor: 08      Soil Sensor: 8%
17:51:29.208 -> Rain sensor: 28      Soil Sensor: 10%
17:51:29.255 -> Rain sensor: 38      Soil Sensor: 10%
17:51:29.302 -> Rain sensor: 38      Soil Sensor: 10%
17:51:29.349 -> Rain sensor: 38      Soil Sensor: 10%
17:51:29.396 -> Rain sensor: 18      Soil Sensor: 10%
17:51:29.443 -> Rain sensor: 38      Soil Sensor: 10%
17:51:29.489 -> Rain sensor: 38      Soil Sensor: 9%
17:51:29.536 -> Rain sensor: 48      Soil Sensor: 9%
17:51:29.583 -> Rain sensor: 38      Soil Sensor: 9%
17:51:29.630 -> Rain sensor: 48      Soil Sensor: 10%
17:51:29.677 -> Rain sensor: 08      Soil Sensor: 8%
17:51:29.724 -> Rain sensor: 18      Soil Sensor: 10%
17:51:29.770 -> Rain sensor: 48      Soil Sensor: 10%
  
```

Fig3.8 Rain sensor and soil moisture sensor reading data

3.6 Conclusion

There are many types of robots that we can see in our daily lives, and each one of them has its own function. Through the research above, many benefits have been concluded that this type of robot can do, as it is multipurpose and its work is not limited to the agricultural side only, but it can be used in many other projects because it is connected to the GPS where it can accomplish any job outside the home. We worked hard for six months to complete this project and we faced many challenges and obstacles, but in the end we were able to produce an integrated project that can be relied upon before it is considered as an educational project to which few new things can be added. To become a project that can be implemented in reality

3.7 References

- [1] Pantazi, X. E., Moshou, D., Alexandridis, T., Whetton, R. L., & Mouazen, A. M. (2016). Wheat yield prediction using machine learning and advanced sensing techniques. *Computers and electronics in agriculture*, 121, 57-65.
- [2] Navrozidis, I., Alexandridis, T. K., Dimitrakos, A., Lagopodi, A. L., Moshou, D., & Zalidis, G. (2018). Identification of purple spot disease on asparagus crops across spatial and spectral scales. *Computers and electronics in Agriculture*, 148, 322-329.
- [3] Vega, F. A., Ramirez, F. C., Saiz, M. P., & Rosua, F. O. (2015). Multi-temporal imaging using an unmanned aerial vehicle for monitoring a sunflower crop. *Biosystems Engineering*, 132, 19-27.
- [4] Pérez-Ortiz, M., Peña, J. M., Gutiérrez, P. A., Torres-Sánchez, J., Hervás-Martínez, C., & López-Granados, F. (2016). Selecting patterns and features for between-and within-crop-row weed mapping using UAV-imagery. *Expert Systems with Applications*, 47, 85-94.
- [5] Tzounis, A., Katsoulas, N., Bartzanas, T., & Kittas, C. (2017). Internet of Things in agriculture, recent advances and future challenges. *Biosystems engineering*, 164, 31-48.
- [6] Siow, E., Tiropanis, T., & Hall, W. (2018). Analytics for the internet of things: A survey. *ACM computing surveys (CSUR)*, 51(4), 1-36.
- [7] Kurnia, Y., & Sie, J. L. (2019). Prototype of Warehouse Automation System Using Arduino Mega 2560 Microcontroller Based on Internet of Things. *bit-Tech*, 1(3), 122-128.
- [8] Dermawan, M., & Meliala, S. (2022). Design Traffic Light of HCSR04 Sensor Fuzzy Logic Method Based on Arduino Mega 2560. *International Journal of Engineering, Science and Information Technology*, 2(4), 133-143.
- [9] Maung, M. M., Latt, M. M., & Nwe, C. M. (2018). DC motor angular position control using PID controller with friction compensation. *International journal of scientific and research publications*, 8(11), 149.
- [10] San, Z. M., Thike, A., & Oo, D. M. (2019). Electronic Forklift by Remote Control. *Int. J. Sci. Eng. Appl*, 8(6), 162-166.
- [11] Chawla, A., Bangera, T., Kolwalkar, C., & Bhat, M. (2015). Bluetooth based weather station. *International Journal of Engineering Trends and Technology (IJETT)*, 28(2), 98-101.
- [12] Jayantilal, S. H. (2014). Interfacing of AT command based HC-05 serial Bluetooth module with minicom in Linux. *International Journal for Scientific Research & Development*, 2(3), 329-332.
- [13] Pham, H. D., Driberg, M., & Nguyen, C. C. (2013, December). Development of vehicle tracking system using GPS and GSM modem. In *2013 IEEE conference on open systems (ICOS)* (pp. 89-94). IEEE.
- [14] Jayandhi, G., Niranjini, K., Revathi, K., Peter, S. J., & Tejaswini, S. (2020, August). DESIGN OF MEDICAL DRONE USING NEO-6M GPS WITH ARDUINO UNO. In *National Conference on Recent Advancements in Communication* (Vol. 7, No. 08).
- [15] Tuan, K. N. (2019, July). A wireless sensor network for aquaculture using Raspberry Pi,

- Arduino and Xbee. In *2019 International Conference on System Science and Engineering (ICSSE)* (pp. 235-238). IEEE.
- [16] Lubis, S., Damanik, W. S., & Siregar, M. A. (2021, January). DESIGN OF QIBLAT DIRECTION USING HMC 5883L SENSOR. In *Proceeding International Seminar of Islamic Studies* (Vol. 2, No. 1, pp. 178-184).
- [17] Abbas, F. N., Saadoon, I. M., Abdalrdha, Z. K., & Abud, E. N. (2020). Capable of gas sensor MQ-135 to monitor the air quality with Arduino Uno. *Int. J. Eng. Res. Technol*, 13(10), 2955-2959.
- [18] Kuria, K. P., Robinson, O. O., & Gabriel, M. M. (2020). Monitoring temperature and humidity using Arduino Nano and Module-DHT11 sensor with real time DS3231 data logger and LCD display. *Health Hyg*, 6(7), 8.
- [19] Dangi, N. (2018). Monitoring environmental parameters: Humidity and temperature using Arduino based microcontroller and sensors: Microcontroller based building monitoring system.
- [20] Al-Obaidi, M. A. M., Radhi, M. A. H., Ibrahim, R. S., & Sutikno, T. (2020). Technique smart control soil moisture system to watering plant based on IoT with arduino uno. *Bulletin of Electrical Engineering and Informatics*, 9(5), 2038-2044.
- [21] Heredia, S., & DeGuia, S. (2018). DC House Solar Panel Cleaner.
- [22] Zhmud, V. A., Kondratiev, N. O., Kuznetsov, K. A., Trubin, V. G., & Dimitrov, L. V. (2018, May). Application of ultrasonic sensor for measuring distances in robotics. In *Journal of Physics: Conference Series* (Vol. 1015, No. 3, p. 032189). IOP Publishing.
- [23] Haidar, A. M., Benachaiba, C., & Zahir, M. (2013). Software interfacing of servo motor with microcontroller.
- [24] Roy, S., Saha, P., & Karmakar, A. DESIGN OF SMART AND WIRELESS IRRIGATION SYSTEM (USING ESP8266 NODEMCU).
- [25] Karthika, A., Jayanthi, S., & Deivamani, G. (2019). Dual axis solar tracking system using Arduino. *International Research Journal of Engineering and Technology (IRJET)*, 6(3), 1034-1036.
- [26] Karlina, F. H., Waruwu, M. M., & Wijaya, R. (2021, December). Study of Several Types of Lithium-polymer Batteries With 3s Battery Management System. In *IOP Conference Series: Earth and Environmental Science* (Vol. 927, No. 1, p. 012023). IOP Publishing.
- [27] Saharia, T., Bauri, J., & Bhagabati, C. (2017). Joystick controlled wheelchair. *Int. Res. J. Eng. Technol*, 4(7), 235-237.
- [28] Kodali, R. K., & Mahesh, K. S. (2016, December). A low cost implementation of MQTT using ESP8266. In *2016 2nd International Conference on Contemporary Computing and Informatics (IC3I)* (pp. 404-408). IEEE.
- [29] Grimstad, L., & From, P. J. (2017). The Thorvald II agricultural robotic system. *Robotics*, 6(4), 24.
- [30] Furlas, G. K., Karras, G. C., & Kyriakopoulos, K. J. (2015). Fault Tolerant Control for a 4-Wheel Skid Steering Mobile Robot. In *DX* (pp. 177-184).
- [31] Louis, L. (2016). working principle of Arduino and u sing it. *International Journal of Control, Automation, Communication and Systems (IJCACS)*, 1(2), 21-29.
- [32] Patton, E. W., Tissenbaum, M., & Harunani, F. (2019). MIT app inventor: Objectives, design, and development. *Computational thinking education*, 31-49.

[33] Bhuiya, K., Anish, K., Parekh, D., & Sahiti, K. L. (2013). Low cost wireless control and monitoring using PLC and SCADA. *International Journal of Scientific and Research Publications*, 3(9), 1-4.